

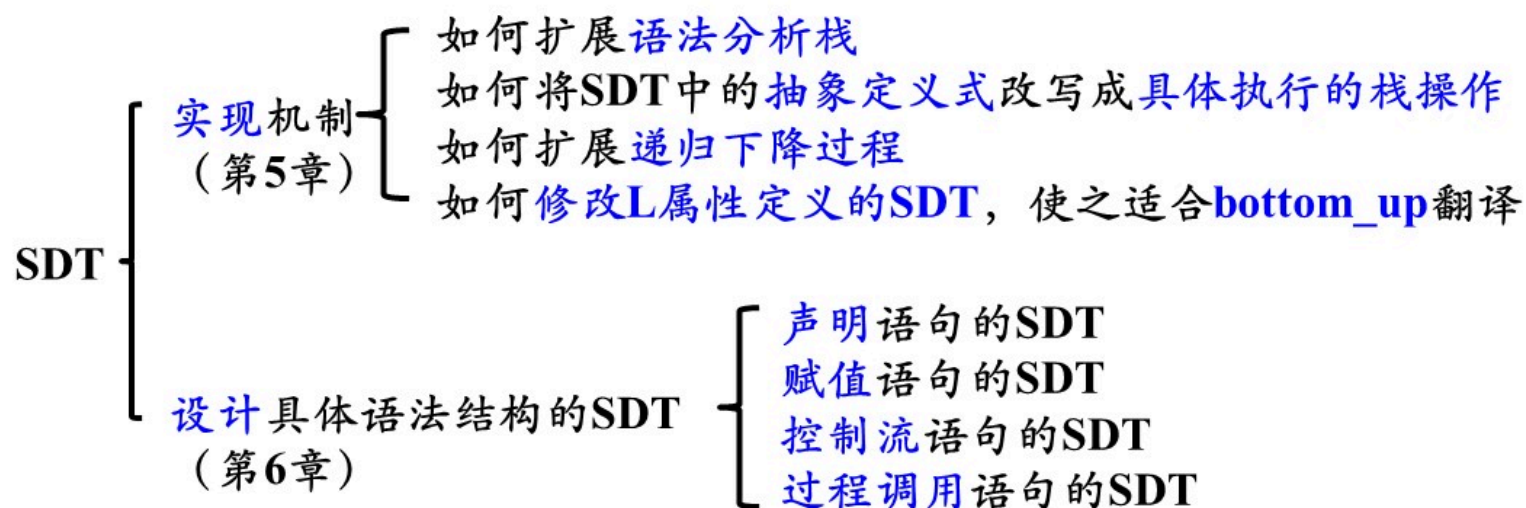
编译原理
第六章
中间代码生成

哈尔滨工业大学 陈鄞



引言

➤ 本章是第5章（语法制导翻译）的延伸





本章内容

6.1 声明语句的翻译

6.2 赋值语句的翻译

6.3 控制语句的翻译

6.4 回填

6.5 switch语句的翻译

6.6 过程调用语句的翻译

6.1 声明语句的翻译

➤ 主要任务

分析所声明id的种属、类型和地址等，在符号表中为id建立一条记录



- 从类型表达式可以知道该类型在运行时刻所需的存储单元数量（称为类型的宽度）
- 在编译时刻，可以使用类型的宽度为每一个名字分配一个相对地址

类型表达式 (*Type Expressions*)

- 基本类型是类型表达式

- *integer*

- *real*

- *char*

- *boolean*

- *type_error* (出错类型)

- *void* (无类型)

类型表达式 (*Type Expressions*)

- 基本类型是类型表达式
- 可以为类型表达式命名，**类型名**也是类型表达式
- 将**类型构造符**(*type constructor*)作用于**类型表达式**可以构成新的类型表达式
 - 数组构造符`array`
 - 若 T 是类型表达式，则`array( $I$ ,  $T$ )`是类型表达式(I 是一个整数)

类型	类型表达式
<code>int [3]</code>	<code>array (3, int)</code>
<code>int [2][3]</code>	<code>array (2, array(3,int))</code>

类型表达式 (*Type Expressions*)

- 基本类型是类型表达式
- 可以为类型表达式命名，**类型名**也是类型表达式
- 将**类型构造符**(*type constructor*)作用于**类型表达式**可以构成新的类型表达式
 - 数组构造符 *array*
 - 指针构造符 *pointer*
 - 若 T 是类型表达式，则 $pointer(T)$ 是类型表达式，它表示一个指针类型

类型表达式 (*Type Expressions*)

- 基本类型是类型表达式
- 可以为类型表达式命名, 类型名也是类型表达式
- 将类型构造符(*type constructor*)作用于类型表达式可以构成新的类型表达式
 - 数组构造符 *array*
 - 指针构造符 *pointer*
 - 笛卡尔乘积构造符 $\times$
 - 若 T_1 和 T_2 是类型表达式, 则笛卡尔乘积 $T_1 \times T_2$ 是类型表达式

类型表达式 (*Type Expressions*)

- 基本类型是类型表达式
- 可以为类型表达式命名，类型名也是类型表达式
- 将类型构造符(*type constructor*)作用于类型表达式可以构成新的类型表达式
 - 数组构造符 *array*
 - 指针构造符 *pointer*
 - 笛卡尔乘积构造符 $\times$
 - 函数构造符 $\rightarrow$
 - 若 T_1 、 T_2 、...、 T_n 和 R 是类型表达式，则 $T_1 \times T_2 \times \dots \times T_n \rightarrow R$ 是类型表达式

类型表达式 (*Type Expressions*)

- 基本类型是类型表达式
- 可以为类型表达式命名, 类型名也是类型表达式
- 将类型构造符(*type constructor*)作用于类型表达式可以构成新的类型表达式
 - 数组构造符 *array*
 - 指针构造符 *pointer*
 - 笛卡尔乘积构造符 $\times$
 - 函数构造符 $\rightarrow$
 - 记录构造符 *record*
 - 若有标识符 $N_1, N_2, \dots, N_n$ 与类型表达式 $T_1, T_2, \dots, T_n$, 则 $record((N_1 \times T_1) \times (N_2 \times T_2) \times \dots \times (N_n \times T_n))$ 是一个类型表达式

例

➤ 设有C程序片段:

```
struct stype
{ char[8] name;
  int score;
};
stype[50] table;
stype* p;
```

- 和*stype*绑定的类型表达式
 - *record* ((*name* × *array*(8, *char*)) × (*score* × *integer*))
- 和*table*绑定的类型表达式
 - *array* (50, *stype*)
- 和*p*绑定的类型表达式
 - *pointer* (*stype*)

局部变量的存储分配

- 对于声明语句，语义分析的主要任务就是收集标识符的**类型**等属性信息，并为每一个名字分配一个**相对地址**
- 从**类型表达式**可以知道该类型在**运行时刻**所需的存储单元数量称为**类型的宽度(width)**
- 在**编译时刻**，可以使用类型的宽度为每一个名字分配一个**相对地址**
- 名字的**类型**和**相对地址**信息保存在相应的**符号表**记录中

变量声明语句的SDT

- ① $P \rightarrow D$
- ② $D \rightarrow T \text{ id}; D$
- ③ $D \rightarrow \varepsilon$
- ④ $T \rightarrow BC$
- ⑤ $T \rightarrow \uparrow T_1$
- ⑥ $B \rightarrow \text{int} \quad \{ B.type = \text{int}; B.width = 4; \}$
- ⑦ $B \rightarrow \text{float} \quad \{ B.type = \text{real}; B.width = 8; \}$
- ⑧ $C \rightarrow \varepsilon$
- ⑨ $C \rightarrow [\text{num}] C_1$

翻译的主要任务：类型表达式和地址

符号	综合属性	
T	$type$	$width$
B	$type$	$width$

变量	作用
$offset$	下一个可用的相对地址

$$offset_{i+1} = offset_i + T_i.width$$

变量声明语句的SDT

翻译的主要任务：类型表达式和地址

① $P \rightarrow \{ offset = 0 \} D$

② $D \rightarrow T \text{ id}; \{ enter(id.lexeme, T.type, offset); offset = offset + T.width; \} D$

③ $D \rightarrow \varepsilon$

④ $T \rightarrow BC$

⑤ $T \rightarrow \uparrow T_1$

⑥ $B \rightarrow \text{int} \{ B.type = \text{int}; B.width = 4; \}$

⑦ $B \rightarrow \text{float} \{ B.type = \text{real}; B.width = 8; \}$

⑧ $C \rightarrow \varepsilon$

⑨ $C \rightarrow [\text{num}] C_1$

符号	综合属性	
T	$type$	$width$
B	$type$	$width$

变量	作用
$offset$	下一个可用的相对地址

$$offset_{i+1} = offset_i + T_i.width$$

变量声明语句的SDT

① $P \rightarrow \{ offset = 0 \} D$

② $D \rightarrow T \text{ id}; \{ enter(\text{id.lexeme}, T.type, offset); offset = offset + T.width; \} D$

③ $D \rightarrow \varepsilon$

④ $T \rightarrow BC$

⑤ $T \rightarrow \uparrow T_1$

⑥ $B \rightarrow \text{int} \{ B.type = integer; B.width = 4; \}$

⑦ $B \rightarrow \text{float} \{ B.type = float; B.width = 8; \}$

⑧ $C \rightarrow \varepsilon$

⑨ $C \rightarrow [\text{num}] C_1$

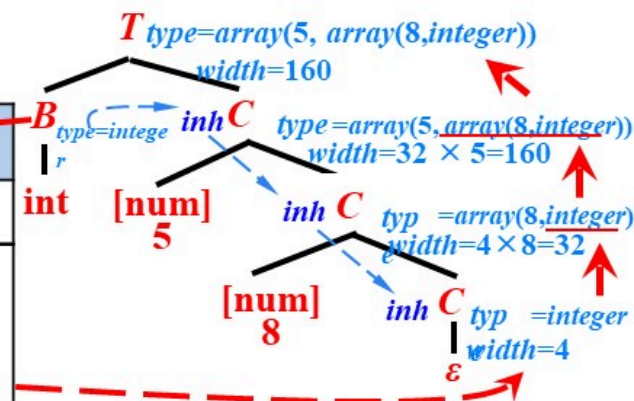
例: `int[5][8]`

`type=array(5, array(8, integer))`

翻译的主要任务: 类型表达式和地址

符号	综合属性	
T	$type$	$width$
B	$type$	$width$
C	$type$	$width$

变量	作用
$offset$	下一个可用的相对地址
t, w	将类型和宽度信息从语法分析树中的 B 结点传递到对应于产生式 $C \rightarrow \varepsilon$ 的结点



变量声明语句的SDT

① $P \rightarrow \{ offset = 0 \} D$

② $D \rightarrow T \text{ id}; \{ enter(\text{id.lexeme}, T.type, offset); offset = offset + T.width; \} D$

③ $D \rightarrow \varepsilon$

④ $T \rightarrow BC$

⑤ $T \rightarrow \uparrow T_1$

⑥ $B \rightarrow \text{int} \{ B.type = \text{integer}; B.width = 4; \}$

⑦ $B \rightarrow \text{float} \{ B.type = \text{float}; B.width = 8; \}$

⑧ $C \rightarrow \varepsilon$

⑨ $C \rightarrow [\text{num}] C_1$

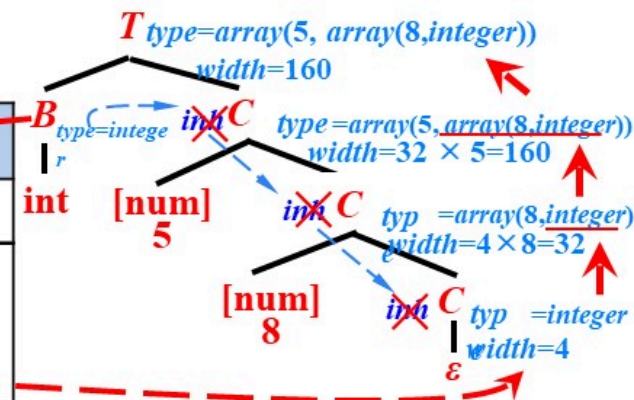
例: `int[5][8]`

`type=array(5, array(8, integer))`

翻译的主要任务: 类型表达式和地址

符号	综合属性	
T	$type$	$width$
B	$type$	$width$
C	$type$	$width$

变量	作用
$offset$	下一个可用的相对地址
t, w	将类型和宽度信息从语法分析树中的 B 结点传递到对应于产生式 $C \rightarrow \varepsilon$ 的结点



变量声明语句的SDT

① $P \rightarrow \{ offset = 0 \} D$

② $D \rightarrow T \text{ id}; \{ enter(id.lexeme, T.type, offset); offset = offset + T.width; \} D$

③ $D \rightarrow \varepsilon$

④ $T \rightarrow B \{ t = B.type; w = B.width; \}$
 $C \{ T.type = C.type; T.width = C.width; \}$

⑤ $T \rightarrow \uparrow T_1 \{ T.type = pointer(T_1.type); T.width = 4; \}$

⑥ $B \rightarrow \text{int} \{ B.type = integer; B.width = 4; \}$

⑦ $B \rightarrow \text{float} \{ B.type = float; B.width = 8; \}$

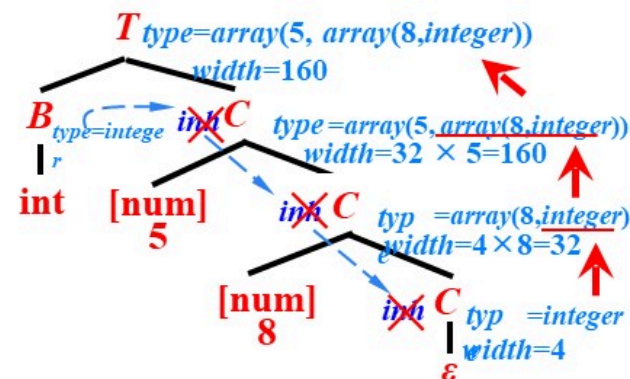
⑧ $C \rightarrow \varepsilon \{ C.type = t; C.width = w; \}$

⑨ $C \rightarrow [\text{num}] C_1 \{ C.type = array(\text{num.val}, C_1.type);$
 $C.width = \text{num.val} * C_1.width; \}$

例: $\text{int}[5][8]$
 $type = array(5, array(8, integer))$

翻译的主要任务: 类型表达式和地址

符号	综合属性	
T	$type$	$width$
B	$type$	$width$
C	$type$	$width$



符号表的组织——数组

➤ 例

int abc;

int i;

int myarray[3][5][8];

名字	基本属性				扩展属性
	种属	类型	地址	扩展属性指针	
abc	变量	int	0	NULL	
i	变量	int	4	NULL	
myarray	数组	int	8		
...	...				

内情向量				
3	3	5	8	← 各维维长
160	32	4		← 各维数组元素的宽度

维数



提纲

6.1 声明语句的翻译

6.2 赋值语句的翻译

6.3 控制语句的翻译

6.4 回填

6.5 switch语句的翻译

6.6 过程调用语句的翻译

6.2 赋值语句的翻译

- 6.2.1 简单赋值语句的翻译
- 6.2.2 数组引用的翻译

6.2.1 简单赋值语句的翻译

➤ 赋值语句的基本文法

① $S \rightarrow \text{id} = E;$

② $E \rightarrow E_1 + E_2$

③ $E \rightarrow E_1 * E_2$

④ $E \rightarrow -E_1$

⑤ $E \rightarrow (E_1)$

⑥ $E \rightarrow \text{id}$

➤ 赋值语句翻译的主要任务

➤ 生成对表达式求值的三地址码

➤ 例

➤ 源程序片段

➤ $x = (a + b) * c;$

➤ 三地址码

➤ $t_1 = a + b$

➤ $t_2 = t_1 * c$

➤ $x = t_2$

临时变量的地址 (常数)

赋值语句的SDT

符号	综合属性
S	$code$
E	$code \quad addr$

$S \rightarrow id = E; \{ p = lookup(id.lexeme); if p == nil then error ;$
 $\quad S.code = E.code \parallel$
 $\quad gen(p '=' E.addr); \}$

$E \rightarrow E_1 + E_2 \{ E.addr = newtemp();$
 $\quad E.code = E_1.code \parallel E_2.code \parallel$
 $\quad gen(E.addr '=' E_1.addr '+' E_2.addr); \}$

newtemp(): 生成一个新的临时变量 t , 返回 t 的地址

$E \rightarrow E_1 * E_2 \{ E.addr = newtemp();$
 $\quad E.code = E_1.code \parallel E_2.code \parallel$
 $\quad gen(E.addr '=' E_1.addr '*' E_2.addr); \}$

gen(code): 生成三地址指令 $code$

$E \rightarrow -E_1 \{ E.addr = newtemp();$
 $\quad E.code = E_1.code \parallel$
 $\quad gen(E.addr '=' 'uminus' E_1.addr); \}$

$E \rightarrow (E_1) \{ E.addr = E_1.addr;$
 $\quad E.code = E_1.code; \}$

lookup(name): 查询符号表 返回 $name$ 对应的记录

$E \rightarrow id \{ E.addr = lookup(id.lexeme); if E.addr == nil then error ;$
 $\quad E.code = "; \}$

增量翻译 (Incremental Translation)

$S \rightarrow id = E;$ { $p = \text{lookup}(id.\text{lexeme});$ if $p == \text{nil}$ then error ;

$S.\text{code} = E.\text{code} \parallel$
 $\text{gen}(p \neq E.\text{addr});$ }

$E \rightarrow E_1 + E_2$ { $E.\text{addr} = \text{newtemp}();$

$E.\text{code} = E_1.\text{code} \parallel E_2.\text{code} \parallel$
 $\text{gen}(E.\text{addr} \neq E_1.\text{addr} \neq E_2.\text{addr});$ }

$E \rightarrow E_1 * E_2$ { $E.\text{addr} = \text{newtemp}();$

$E.\text{code} = E_1.\text{code} \parallel E_2.\text{code} \parallel$
 $\text{gen}(E.\text{addr} \neq E_1.\text{addr} \neq E_2.\text{addr});$ }

$E \rightarrow -E_1$ { $E.\text{addr} = \text{newtemp}();$

$E.\text{code} = E_1.\text{code} \parallel$
 $\text{gen}(E.\text{addr} \neq \text{'uminus'} E_1.\text{addr});$ }

$E \rightarrow (E_1)$ { $E.\text{addr} = E_1.\text{addr};$

$E.\text{code} = E_1.\text{code};$ }

$E \rightarrow id$ { $E.\text{addr} = \text{lookup}(id.\text{lexeme});$ if $E.\text{addr} == \text{nil}$ then error ;

$E.\text{code} = \text{' '};$ }

在增量方法中， $\text{gen}()$ 不仅要构造出一个新的三地址指令，还要将它添加到至今为止已生成的指令序列之后

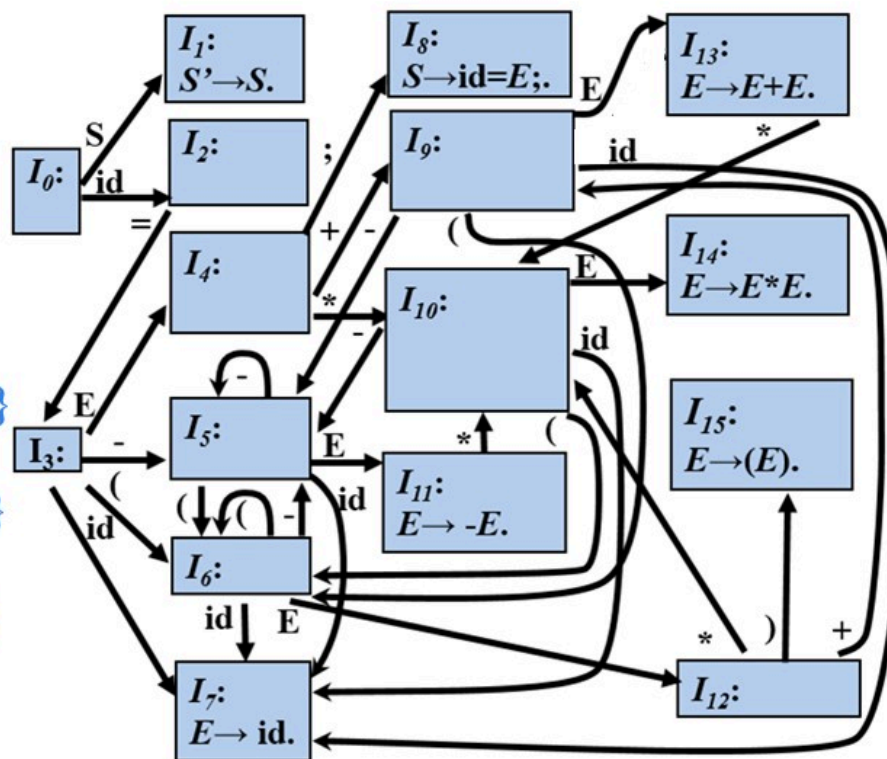
例

- ① $S \rightarrow id = E;$ { $p = lookup(id.lexeme);$
if $p == nil$ then error ;
 $gen(p \neq E.addr);$ }
- ② $E \rightarrow E_1 + E_2$ { $E.addr = newtemp();$
 $gen(E.addr \neq E_1.addr \text{ '+' } E_2.addr);$ }
- ③ $E \rightarrow E_1 * E_2$ { $E.addr = newtemp();$
 $gen(E.addr \neq E_1.addr \text{ '*' } E_2.addr);$ }
- ④ $E \rightarrow -E_1$ { $E.addr = newtemp();$
 $gen(E.addr \neq \text{'uminus' } E_1.addr);$ }
- ⑤ $E \rightarrow (E_1)$ { $E.addr = E_1.addr;$ }
- ⑥ $E \rightarrow id$ { $E.addr = lookup(id.lexeme);$
if $E.addr == nil$ then error ; }

例: $x = (a + b) * c;$



0	2	3	6	7
\$	id	=	(	id
	x			a



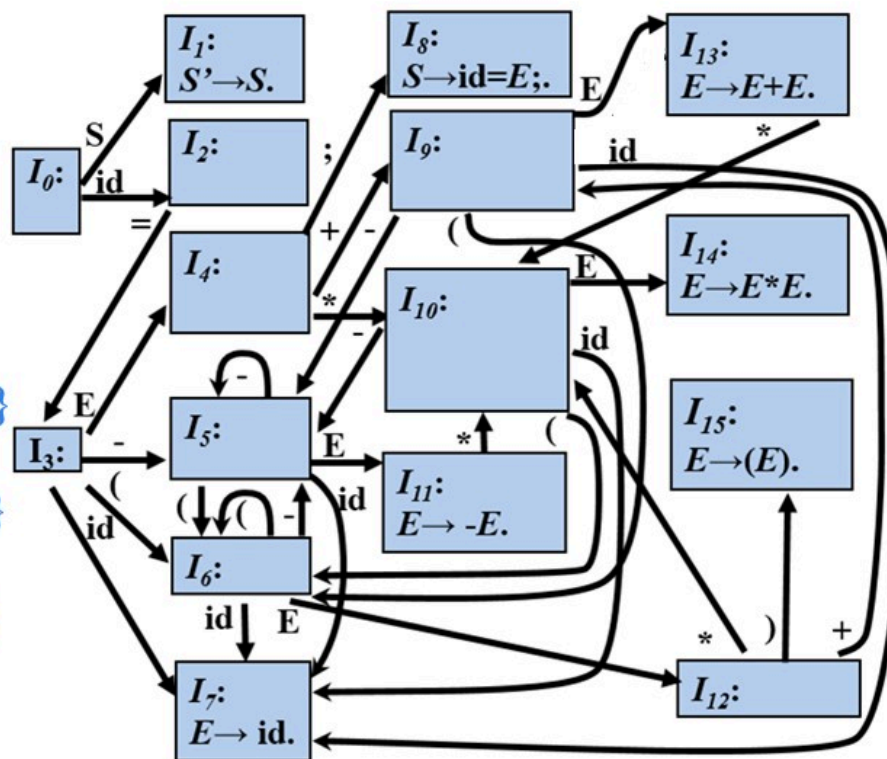
例

- ① $S \rightarrow id = E;$ { $p = lookup(id.lexeme);$
if $p == nil$ then error ;
gen($p \neq E.addr$); }
- ② $E \rightarrow E_1 + E_2$ { $E.addr = newtemp();$
gen($E.addr \neq E_1.addr + E_2.addr$); }
- ③ $E \rightarrow E_1 * E_2$ { $E.addr = newtemp();$
gen($E.addr \neq E_1.addr * E_2.addr$); }
- ④ $E \rightarrow -E_1$ { $E.addr = newtemp();$
gen($E.addr \neq 'uminus' E_1.addr$); }
- ⑤ $E \rightarrow (E_1)$ { $E.addr = E_1.addr ;$ }
- ⑥ $E \rightarrow id$ { $E.addr = lookup(id.lexeme);$
if $E.addr == nil$ then error ; }

例: $x = (a + b) * c ;$



0	2	3	6	12	9	7
\$	id	=	(	E	+	id
	x			a		b



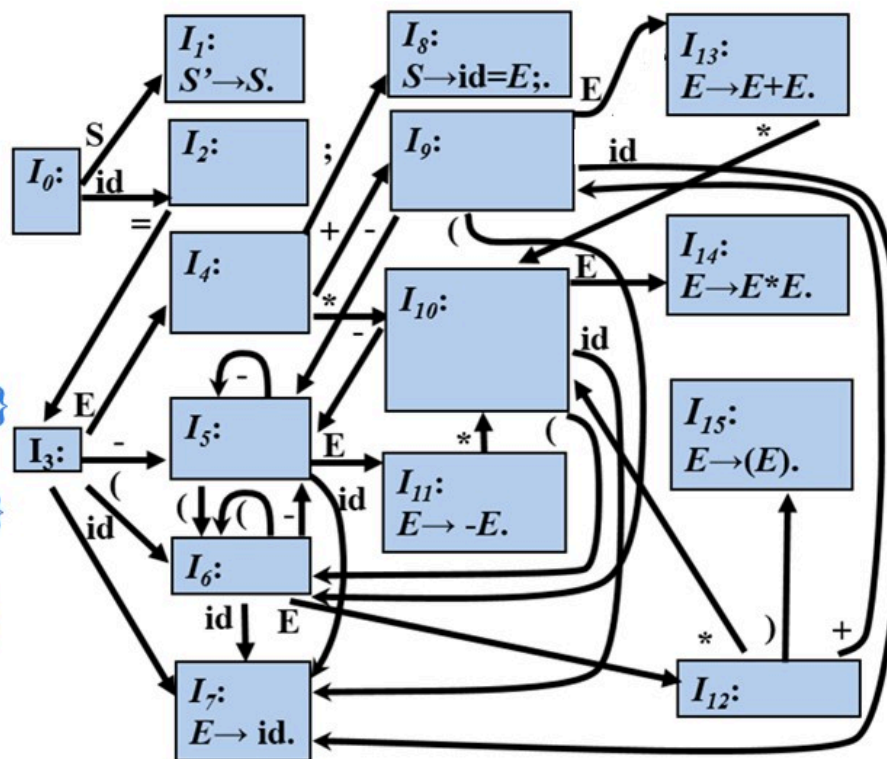
例

- ① $S \rightarrow id = E;$ { $p = lookup(id.lexeme);$
if $p == nil$ then error ;
gen($p \neq E.addr$); }
- ② $E \rightarrow E_1 + E_2$ { $E.addr = newtemp();$
gen($E.addr \neq E_1.addr + E_2.addr$); }
- ③ $E \rightarrow E_1 * E_2$ { $E.addr = newtemp();$
gen($E.addr \neq E_1.addr * E_2.addr$); }
- ④ $E \rightarrow -E_1$ { $E.addr = newtemp();$
gen($E.addr \neq 'uminus' E_1.addr$); }
- ⑤ $E \rightarrow (E_1)$ { $E.addr = E_1.addr;$ }
- ⑥ $E \rightarrow id$ { $E.addr = lookup(id.lexeme);$
if $E.addr == nil$ then error ; }

例: $x = (a + b) * c;$



0	2	3	6	12	9	13
\$	id	=	(	E	+	E
	x			a		b



$t_1 = a + b$

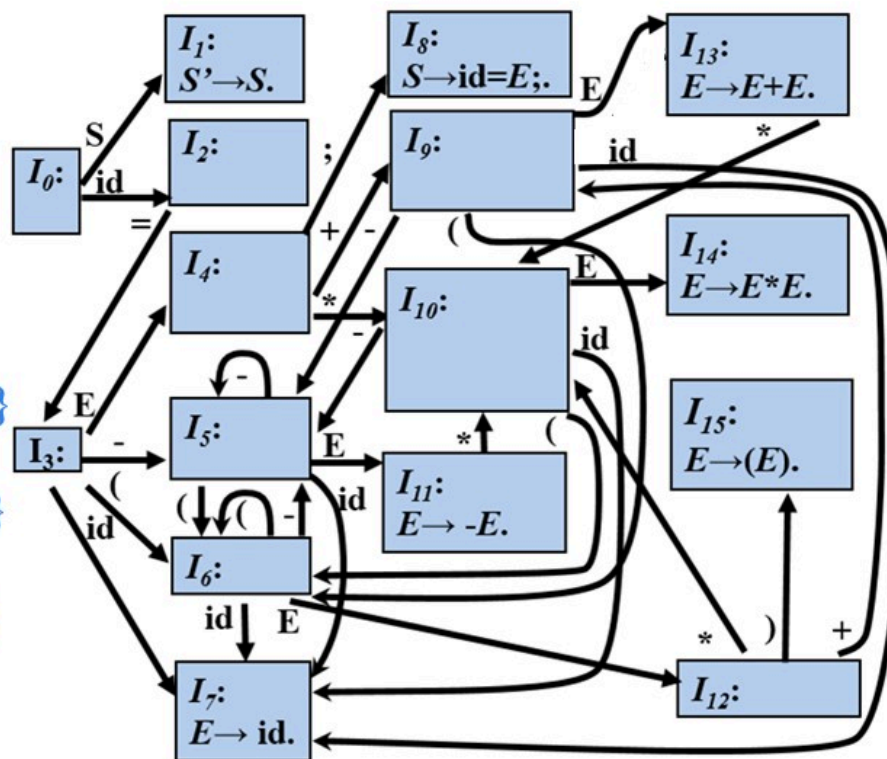
例

- ① $S \rightarrow id = E; \{ p = lookup(id.lexeme);$
 $\text{if } p == nil \text{ then error ;}$
 $\text{gen}(p \neq E.addr); \}$
- ② $E \rightarrow E_1 + E_2 \{ E.addr = newtemp();$
 $\text{gen}(E.addr \neq E_1.addr \text{ '+' } E_2.addr); \}$
- ③ $E \rightarrow E_1 * E_2 \{ E.addr = newtemp();$
 $\text{gen}(E.addr \neq E_1.addr \text{ '*' } E_2.addr); \}$
- ④ $E \rightarrow -E_1 \{ E.addr = newtemp();$
 $\text{gen}(E.addr \neq \text{'uminus' } E_1.addr); \}$
- ⑤ $E \rightarrow (E_1) \{ E.addr = E_1.addr ; \}$
- ⑥ $E \rightarrow id \{ E.addr = lookup(id.lexeme);$
 $\text{if } E.addr == nil \text{ then error ;} \}$

例: $x = (a + b) * c;$



0	2	3	6	12	15
\$	id	=	(	E	)
x			t ₁		



$t_1 = a + b$

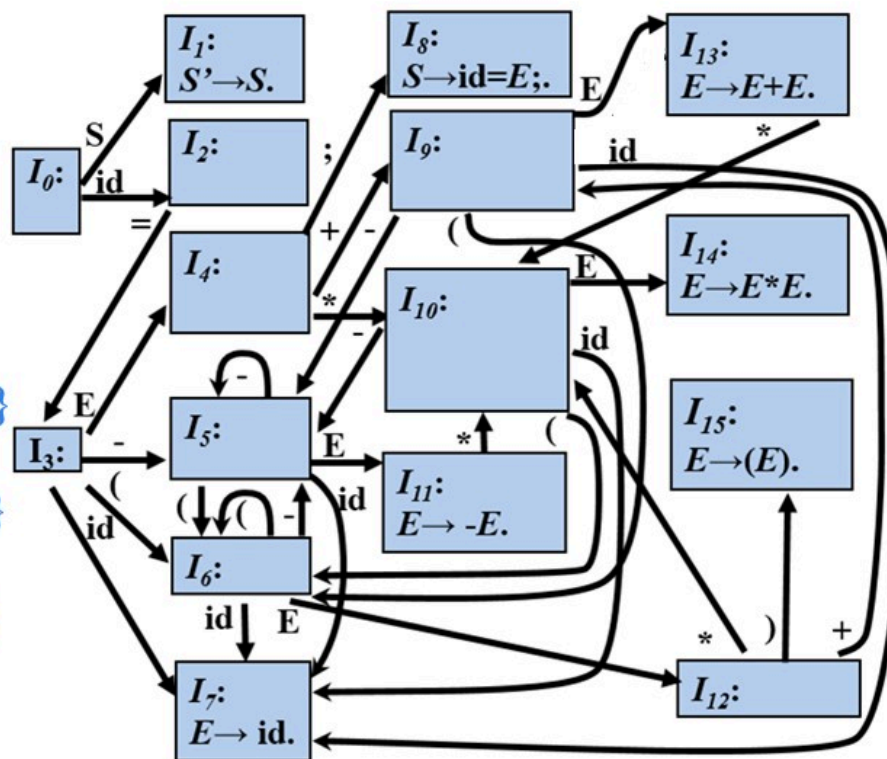
例

- ① $S \rightarrow id = E;$ { $p = lookup(id.lexeme);$
if $p == nil$ then error ;
gen($p \neq E.addr$); }
- ② $E \rightarrow E_1 + E_2$ { $E.addr = newtemp();$
gen($E.addr \neq E_1.addr + E_2.addr$); }
- ③ $E \rightarrow E_1 * E_2$ { $E.addr = newtemp();$
gen($E.addr \neq E_1.addr * E_2.addr$); }
- ④ $E \rightarrow -E_1$ { $E.addr = newtemp();$
gen($E.addr \neq 'uminus' E_1.addr$); }
- ⑤ $E \rightarrow (E_1)$ { $E.addr = E_1.addr;$ }
- ⑥ $E \rightarrow id$ { $E.addr = lookup(id.lexeme);$
if $E.addr == nil$ then error ; }

例: $x = (a + b) * c;$



0	2	3	4	10	7
\$	id	=	E	*	id
	x		t_1		c



$$t_1 = a + b$$

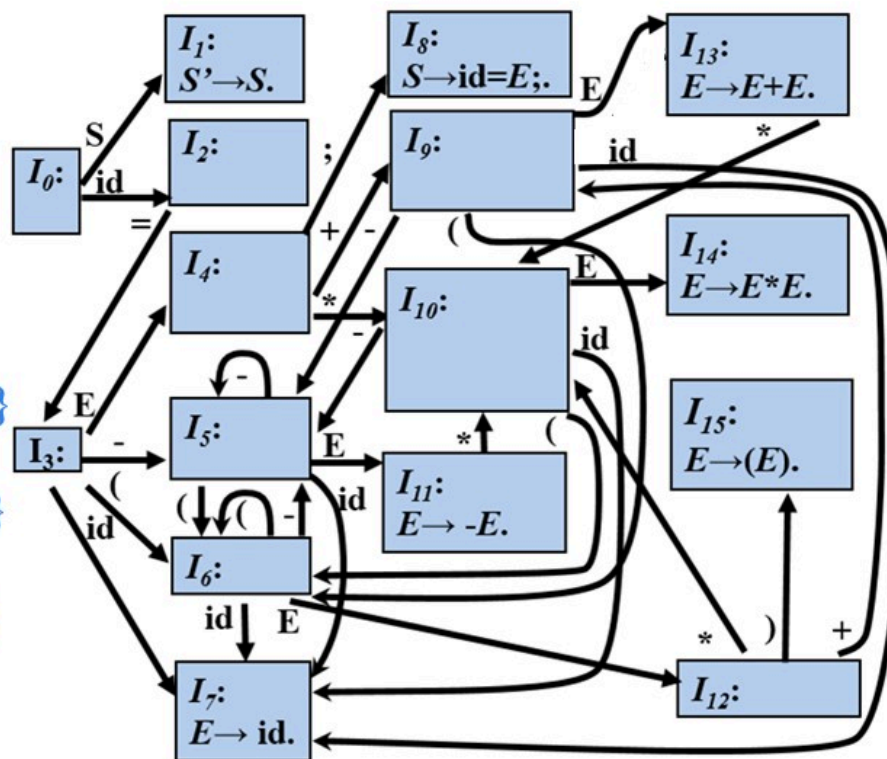
例

- ① $S \rightarrow id = E;$ { $p = lookup(id.lexeme);$
if $p == nil$ then error ;
gen($p \neq E.addr$); }
- ② $E \rightarrow E_1 + E_2$ { $E.addr = newtemp();$
gen($E.addr \neq E_1.addr + E_2.addr$); }
- ③ $E \rightarrow E_1 * E_2$ { $E.addr = newtemp();$
gen($E.addr \neq E_1.addr * E_2.addr$); }
- ④ $E \rightarrow -E_1$ { $E.addr = newtemp();$
gen($E.addr \neq 'uminus' E_1.addr$); }
- ⑤ $E \rightarrow (E_1)$ { $E.addr = E_1.addr;$ }
- ⑥ $E \rightarrow id$ { $E.addr = lookup(id.lexeme);$
if $E.addr == nil$ then error ; }

例: $x = (a + b) * c;$



0	2	3	4	10	14
\$	id	=	E	*	E
	x		t ₁		c



$$t_1 = a + b$$

$$t_2 = t_1 * c$$

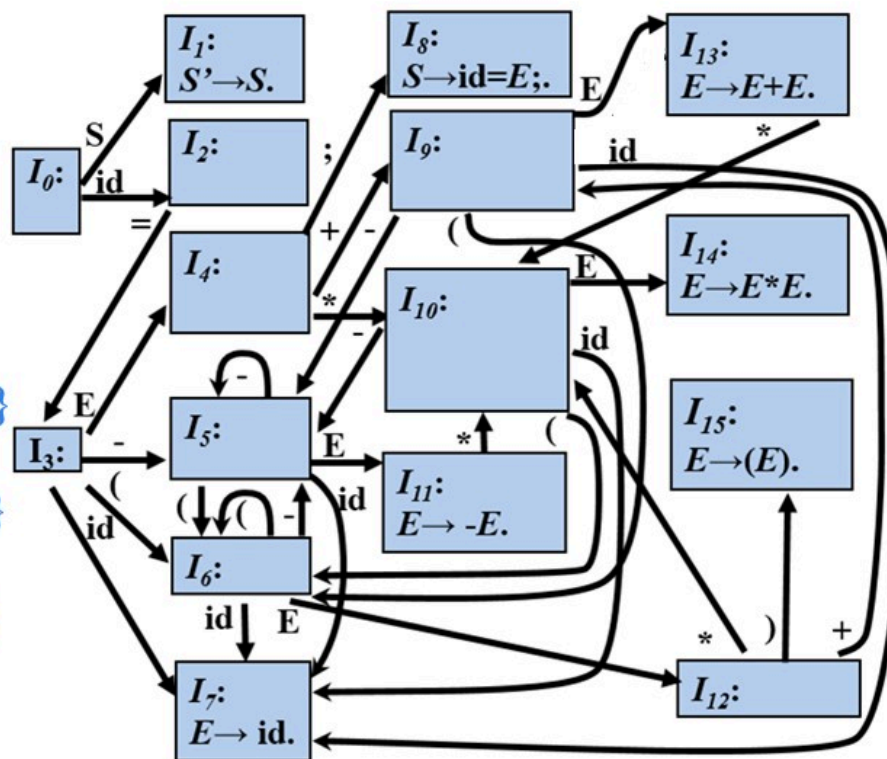
例

- ① $S \rightarrow id = E;$ { $p = lookup(id.lexeme);$
if $p == nil$ then error ;
gen($p \neq E.addr$); }
- ② $E \rightarrow E_1 + E_2$ { $E.addr = newtemp();$
gen($E.addr \neq E_1.addr + E_2.addr$); }
- ③ $E \rightarrow E_1 * E_2$ { $E.addr = newtemp();$
gen($E.addr \neq E_1.addr * E_2.addr$); }
- ④ $E \rightarrow -E_1$ { $E.addr = newtemp();$
gen($E.addr \neq 'uminus' E_1.addr$); }
- ⑤ $E \rightarrow (E_1)$ { $E.addr = E_1.addr ;$ }
- ⑥ $E \rightarrow id$ { $E.addr = lookup(id.lexeme);$
if $E.addr == nil$ then error ; }

例: $x = (a + b) * c ;$



0	2	3	4	8
\$	id	=	E	;
	x		t ₂	



$$t_1 = a + b$$

$$t_2 = t_1 * c$$

$$x = t_2$$

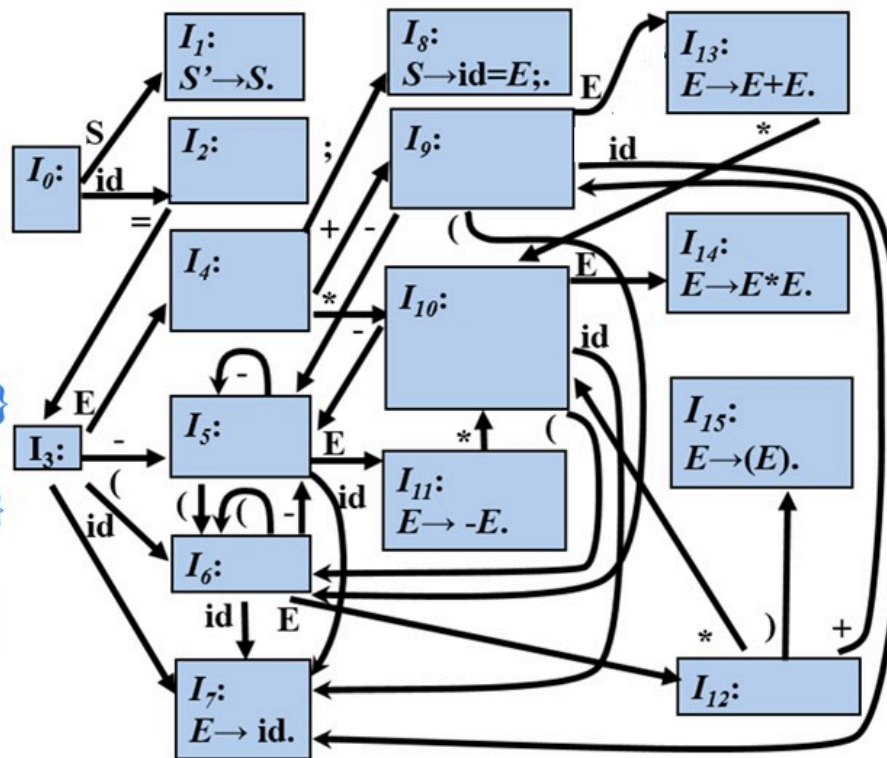
例

- ① $S \rightarrow id = E;$ { $p = lookup(id.lexeme);$
if $p == nil$ then error ;
 $gen(p \neq E.addr);$ }
- ② $E \rightarrow E_1 + E_2$ { $E.addr = newtemp();$
 $gen(E.addr \neq E_1.addr \text{ '+' } E_2.addr);$ }
- ③ $E \rightarrow E_1 * E_2$ { $E.addr = newtemp();$
 $gen(E.addr \neq E_1.addr \text{ '*' } E_2.addr);$ }
- ④ $E \rightarrow -E_1$ { $E.addr = newtemp();$
 $gen(E.addr \neq \text{'uminus' } E_1.addr);$ }
- ⑤ $E \rightarrow (E_1)$ { $E.addr = E_1.addr;$ }
- ⑥ $E \rightarrow id$ { $E.addr = lookup(id.lexeme);$
if $E.addr == nil$ then error ; }

例: $x = (a + b) * c;$



0	1
\$	S



$$t_1 = a + b$$

$$t_2 = t_1 * c$$

$$x = t_2$$

上一讲内容回顾

- 6.1 声明语句的翻译
- 6.2 赋值语句的翻译
 - 6.2.1 简单赋值语句的翻译
 - 6.2.2 数组引用的翻译

6.2.2 数组引用 (*array reference*) 的翻译

➤ 赋值语句的文法

$$S \rightarrow \text{id} = E; \mid L = E;$$
$$E \rightarrow E_1 + E_2 \mid -E_1 \mid (E_1) \mid \text{id} \mid L$$
$$L \rightarrow \text{id} [E] \mid L_1 [E]$$

将数组引用翻译成三地址码时要解决的主要问题是确定数组元素的存放地址，也就是数组元素的寻址

数组元素寻址 (*Addressing Array Elements*)

➤ 一维数组

- 假设每个数组元素的宽度是 w ，则数组元素 $a[i]$ 的相对地址是：

$$base + i \times w$$

其中， $base$ 是数组的**基地址**， $i \times w$ 是**偏移量**

➤ 二维数组

- 假设一行的宽度是 w_1 ，同一行中每个数组元素的宽度是 w_2 ，则数组元素 $a[i_1][i_2]$ 的相对地址是：

$$base + \underbrace{i_1 \times w_1 + i_2 \times w_2}_{\text{偏移量}}$$

➤ k 维数组

- 数组元素 $a[i_1][i_2] \dots [i_k]$ 的相对地址是：

$$base + \underbrace{i_1 \times w_1 + i_2 \times w_2 + \dots + i_k \times w_k}_{\text{偏移量}}$$

$w_1 \rightarrow k-1$ 维数组 $a[i_1]$ 的宽度

$w_2 \rightarrow k-2$ 维数组 $a[i_1][i_2]$ 的宽度

...

$w_k \rightarrow k-k$ 维数组 $a[i_1][i_2] \dots [i_k]$ 的宽度

例

➤ 假设 $\text{type}(a) = \text{array}(3, \text{array}(5, \text{array}(8, \text{integer})))$,
一个整型变量占用4个字节,

$$\begin{aligned}\text{则 } \text{addr}(a[i_1][i_2][i_3]) &= \text{base} + i_1 * w_1 + i_2 * w_2 + i_3 * w_3 \\ &= \text{base} + i_1 * \underline{160} + i_2 * \underline{32} + i_3 * \underline{4}\end{aligned}$$

k-1维数组 $a[i_1]$ 的宽度

k-2维数组 $a[i_1][i_2]$ 的宽度

k-3维数组 $a[i_1][i_2][i_3]$ 的宽度

带有数组引用的赋值语句的翻译

➤ 例

假设 $type(a) = array(5, array(8 \text{ integer}))$

➤ 源程序片段

➤ $c = a[i_1][i_2];$

$$addr(a[i_1][i_2]) = base + \underbrace{i_1 * 32 + i_2 * 4}_{\text{offset}}$$

➤ 三地址码

➤ $t_1 = i_1 * 32$

➤ $t_2 = i_2 * 4$

➤ $t_3 = t_1 + t_2$

➤ $t_4 = a[t_3]$

➤ $c = t_4$

$S \rightarrow id = E; \{ p = lookup(id.lexeme);$
 $\quad if p == nil \text{ then error};$
 $\quad gen(p \neq E.addr); \}$

$E \rightarrow E_1 + E_2 \mid -E_1 \mid (E_1) \mid id \mid L$

$L \rightarrow id [E] \mid L_1 [E]$

数组元素寻址的SDT

➤ 赋值语句的基本文法

$S \rightarrow \text{id} = E; \mid L = E;$

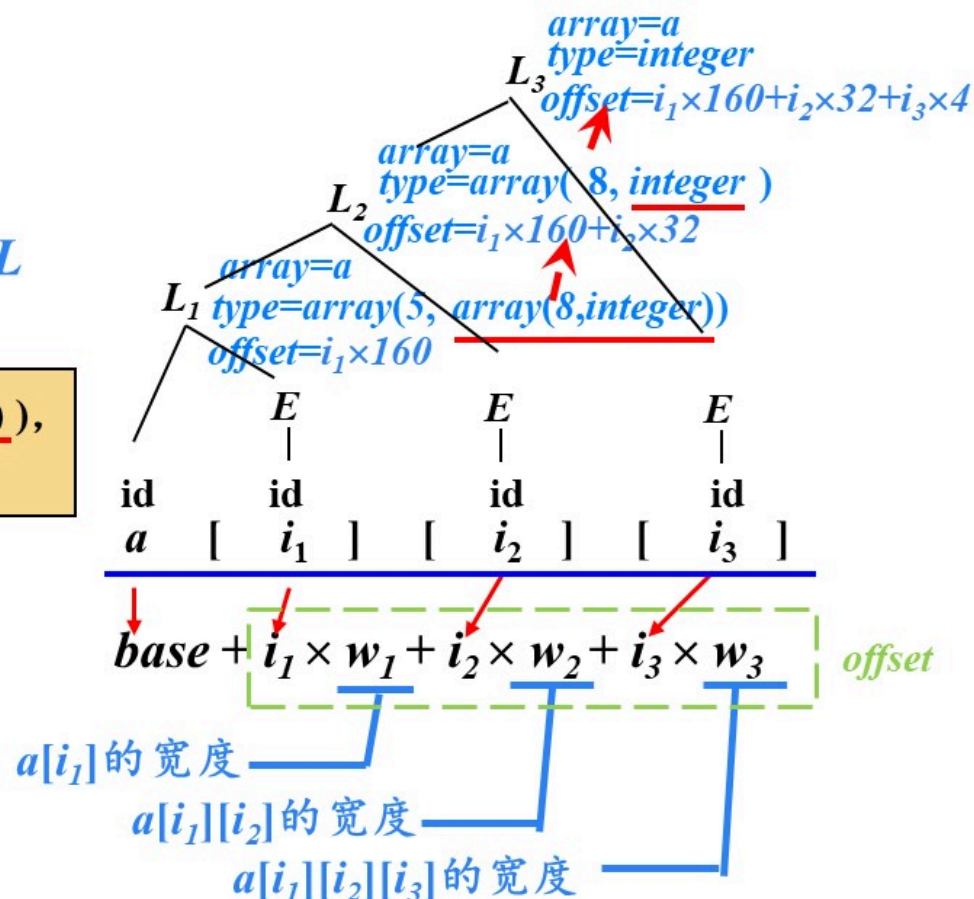
$E \rightarrow E_1 + E_2 \mid -E_1 \mid (E_1) \mid \text{id} \mid L$

$L \rightarrow \text{id} [E] \mid L_1 [E]$

假设 $\text{type}(a) = \text{array}(3, \text{array}(5, \text{array}(8, \text{integer})))$,
翻译语句片段 “ $a[i_1][i_2][i_3]$ ”

➤ L 的综合属性

- $L.\text{type}$: L 生成的数组元素的类型
- $L.\text{offset}$: 指示一个临时变量, 该临时变量用于累加公式中的 $i_j \times w_j$ 项, 从而计算数组元素的偏移量
- $L.\text{array}$: 数组名在符号表的入口地址



数组元素寻址的SDT

假设 $type(a) = array(3, array(5, array(8, int)))$,
翻译语句片段 “ $a[i_1][i_2][i_3]$ ”

$S \rightarrow id = E;$

$| L = E; \{ gen(L.array.base '[' L.offset ']' '=' E.addr); \}$

$E \rightarrow E_1 + E_2 | -E_1 | (E_1) | id$

$| L \{ E.addr = newtemp(); gen(E.addr '=' L.array.base '[' L.offset ']'); \}$

$L \rightarrow id [E] \{ L.array = lookup(id.lexeme); if L.array == nil then error ;$

$L.type = L.array.type.elem ;$

$L.offset = newtemp();$

$gen(L.offset '=' E.addr '*' L.type.width); \}$

$| L_1[E] \{ L.array = L_1.array;$

$L.type = L_1.type.elem ;$

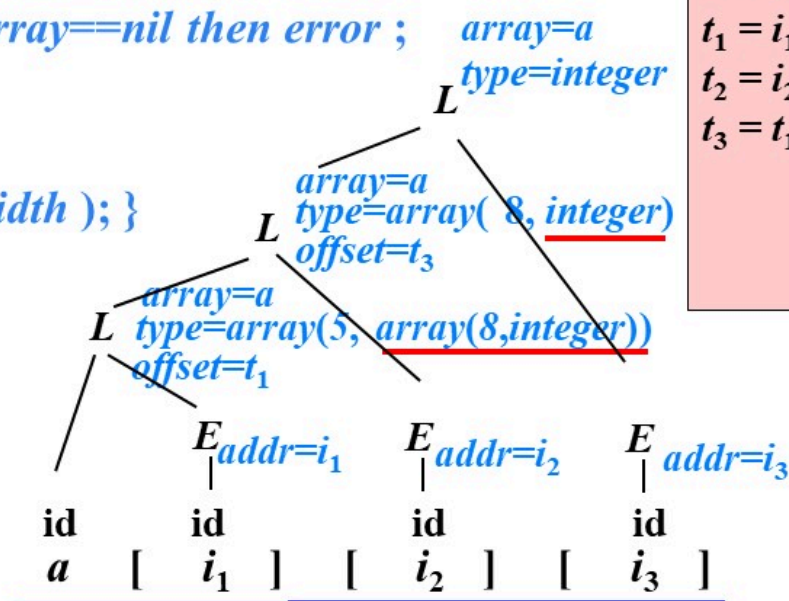
$t = newtemp();$

$gen(t '=' E.addr '*' L.type.width);$

$L.offset = newtemp();$

$gen(L.offset '=' L_1.offset '+' t); \}$

$$addr(a[i_1][i_2][i_3]) = \underbrace{base}_{t_1} + \underbrace{i_1 \times w_1}_{t_2} + \underbrace{i_2 \times w_2}_{t_3} + \underbrace{i_3 \times w_3}_{t_4}$$



三地址码

$$t_1 = i_1 * 160$$

$$t_2 = i_2 * 32$$

$$t_3 = t_1 + t_2$$

假设 `type(a)=array(3, array(5, array(8, int) ) )`,
翻译语句片段 “`a[i1][i2][i3]`”

$$S \rightarrow \text{id} = E;$$

```
| L = E; { gen( L.array.base '[' L.offset ']' '=' E.addr ); }
```

$$E \rightarrow E_1 + E_2 \mid -E_1 \mid (E_1) \mid \mathbf{id}$$

```
L { E.addr = newtemp(); gen( E.addr '=' L.array.base '[' L.offset ']' ); }
```

$$L \rightarrow \text{id } [E] \{ L.array = \text{lookup}(\text{id.lexeme}); \text{ if } L.array == \text{nil} \text{ then error ;}$$

```
L.type = L.array.type.elem ;
```

```
L.offset = newtemp();
```

```
gen( L.offset '=' E.addr '*' L.type.width ); }
```

$$|L_1[E]\{L.array = L_1.array;$$
$$L.type = L_1.type.elem ;$$

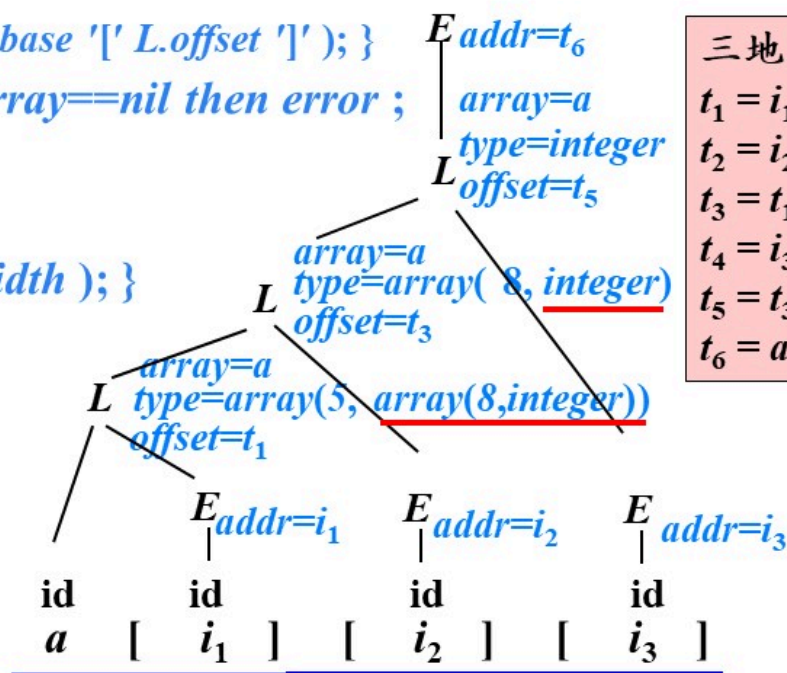
```
t = newtemp();
```

```
gen( t '=' E.addr '*' L.type.width );
```

```
L.offset = newtemp();
```

```
gen( L.offset '=' L1.offset '+' t ); }
```

$$addr(a[i_1][i_2][i_3]) = base + \underbrace{i_1 \times w_1}_{t_1} + \underbrace{i_2 \times w_2}_{t_2} + \underbrace{i_3 \times w_3}_{t_4}$$



三地址码

$$t_1 = i_1 * 160$$

$$t_j = i_j * 32$$

$$t_3 = t_1 + t_2$$

$$t_4 = i_3 * 4$$

$$t_5 = t_3 + t_4$$

$$t_6 = a[t_5]$$

数组元素寻址的SDT

假设 $type(a) = array(3, array(5, array(8, int)))$,
翻译语句片段 “ $a[i_1][i_2][i_3]$ ”

$$addr(a[i_1][i_2][i_3]) = base + \underbrace{i_1 \times w_1}_{t_1} + \underbrace{i_2 \times w_2}_{t_2} + \underbrace{i_3 \times w_3}_{t_3}$$

$S \rightarrow id = E;$

$| L = E; \{ gen(L.array.base '[' L.offset ']' '=' E.addr); \}$

$E \rightarrow E_1 + E_2 | -E_1 | (E_1) | id$

$| L \{ E.addr = newtemp(); gen(E.addr '=' L.array.base '[' L.offset ']'); \}$

$L \rightarrow id [E] \{ L.array = lookup(id.lexeme); if L.array == nil then error ;$

$L.type = L.array.type.elem ;$

$L.offset = newtemp();$

$gen(L.offset '=' E.addr '*' L.type.width); \}$

$| L_1[E] \{ L.array = L_1.array;$

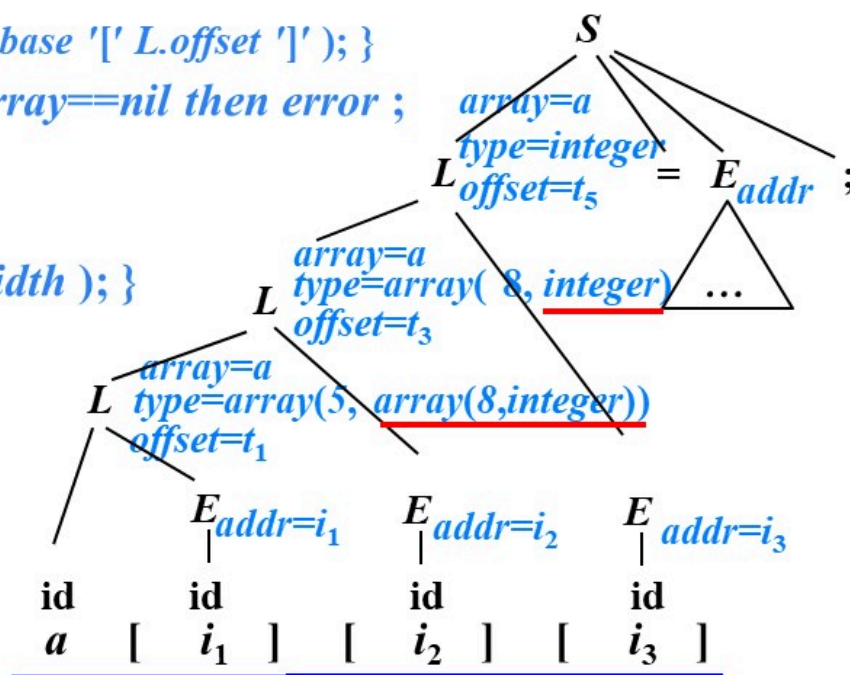
$L.type = L_1.type.elem ;$

$t = newtemp();$

$gen(t '=' E.addr '*' L.type.width);$

$L.offset = newtemp();$

$gen(L.offset '=' L_1.offset '+' t); \}$



总结

假设 $type(a) = array(3, array(5, array(8, int)))$,
翻译语句片段 “ $a[i_1][i_2][i_3]$ ”

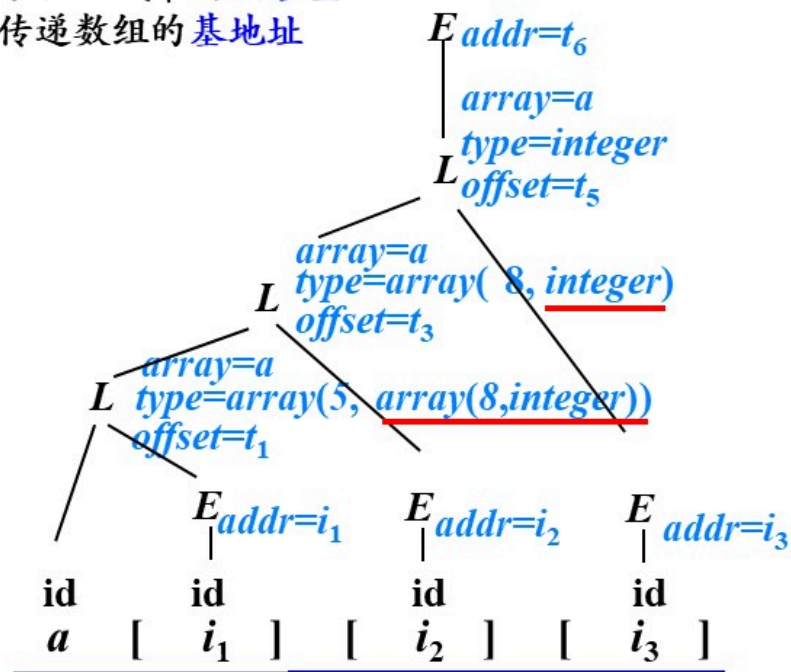
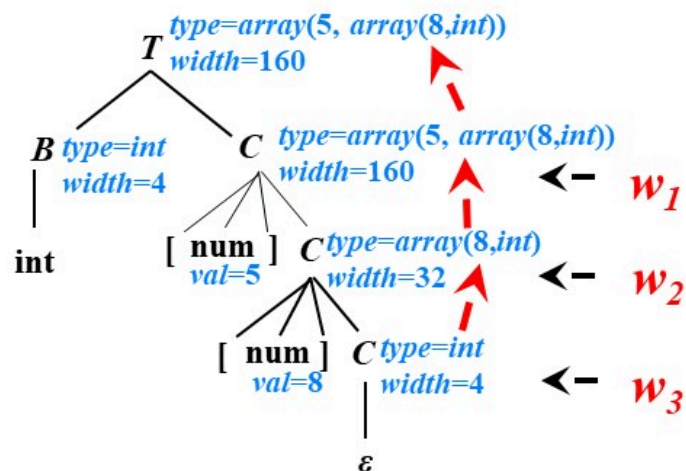
翻译声明语句时已经计算过 $addr(a[i_1][i_2][i_3]) = base + \underbrace{i_1 \times w_1}_{t_1} + \underbrace{i_2 \times w_2}_{t_2} + \underbrace{i_3 \times w_3}_{t_3}$

设置 $type$ 属性: 计算宽度 w_i

设置 $offset$ 属性: 累积公式中的偏移量

设置 $array$ 属性: 传递数组的基地址

数组声明语句的翻译



符号表的组织——数组

➤ 例

```
int abc;
```

```
int i;
```

```
int myarray[3][5][8];
```

名字	基本属性				扩展属性
	种属	类型	地址	扩展属性指针	
abc	变量	int	0	NULL	
i	变量	int	4	NULL	
myarray	数组	int	8		
...	...				

维数 各维维长

3	3	5	8
	160	32	4

各维数组元素的宽度

内情向量

数组元素寻址的SDT

假设 `type(a)=array(3, array(5, array(8, int) ) )`,
翻译语句片段 “`a[i1][i2][i3]`”

$$S \rightarrow \text{id} = E;$$

```
| L = E; { gen( L.array.base '[' L.offset ']' '=' E.addr ); }
```

$$E \rightarrow E_1 + E_2 \mid -E_1 \mid (E_1) \mid \text{id}$$

```
L { E.addr = newtemp(); gen( E.addr '=' L.array.base '[' L.offset ']' ); }
```

$L \rightarrow \text{id } [E] \{ L.array = \text{lookup}(\text{id.lexeme}); \text{if } L.array == \text{nil} \text{ then error} ;$

L.type = L.array.type.elem ;

```
L.offset = newtemp();
```

```
gen( L.offset == E.addr '*' L.type.width ); }
```

$$|L_1[E]\{L.array = L_1.array;$$
$$L.type = L_1.type.elem ;$$

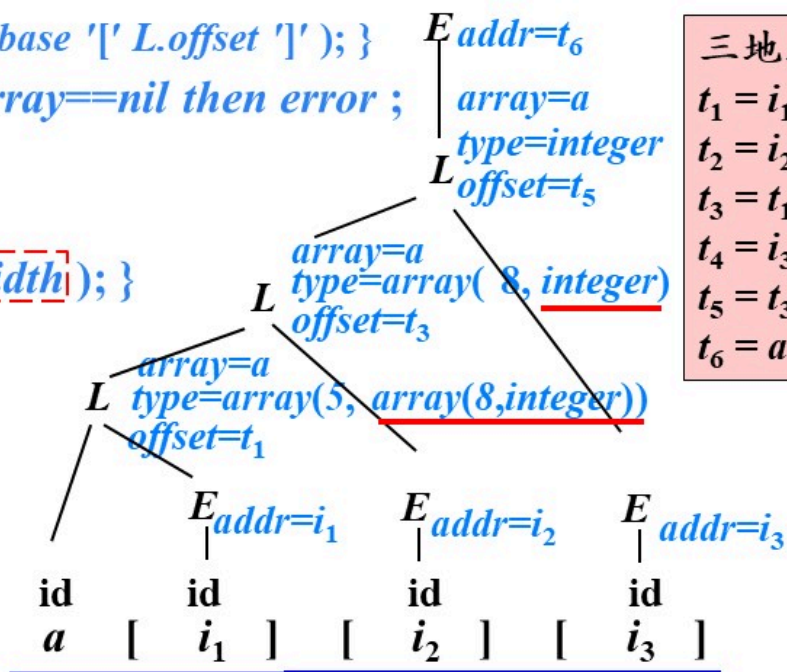
```
t = newtemp();
```

```
gen( t '=' E.addr '*' L.type.width );
```

```
L.offset = newtemp();
```

```
gen( L.offset '=' L1.offset '+' t ); }
```

$$addr(a[i_1][i_2][i_3]) = base + \underbrace{i_1 \times w_1}_{t_1} + \underbrace{i_2 \times w_2}_{t_2} + \underbrace{i_3 \times w_3}_{t_3 \neq t_4}$$



三地址码

$$t_1 = i_1 * 160$$

$$t_2 = i_2 * 32$$

$$t_3 = t_1 + t_2$$

$$t_4 = i_3 * 4$$

$$t_5 = t_3 + t_4$$

$$t_6 = a[t_5]$$



提纲

6.1 声明语句的翻译

6.2 赋值语句的翻译

6.3 控制语句的翻译

6.4 回填

6.5 switch语句的翻译

6.6 过程调用语句的翻译

6.3 控制语句的翻译

➤ 控制流语句的基本文法

➤ $P \rightarrow S$

➤ $S \rightarrow S_1 S_2$

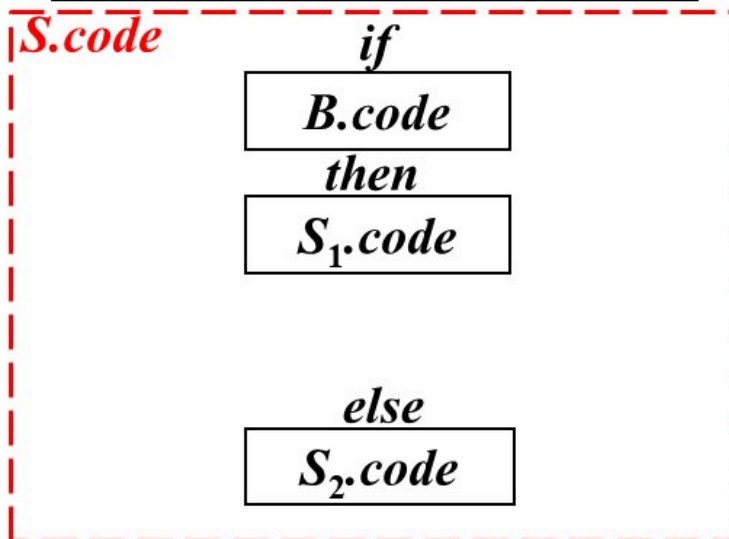
➤ $S \rightarrow \text{id} = E ; \mid L = E ;$

➤ $S \rightarrow \text{if } B \text{ then } S_1$
 $\mid \text{if } B \text{ then } S_1 \text{ else } S_2$
 $\mid \text{while } B \text{ do } S_1$

控制流语句的代码结构

➤ 例

$S \rightarrow \text{if } B \text{ then } S_1 \text{ else } S_2$



例1: $B = "a < b"$

三地址指令

if a < b goto $S_1.first$
goto $S_2.first$

标号(常量)

问题: 生成跳转指令的时候, $S_1.first$ 和 $S_2.first$ 的值可能还不知道

例2: $B = "a < b \parallel a > 200 \ \&\& \ b < 100"$

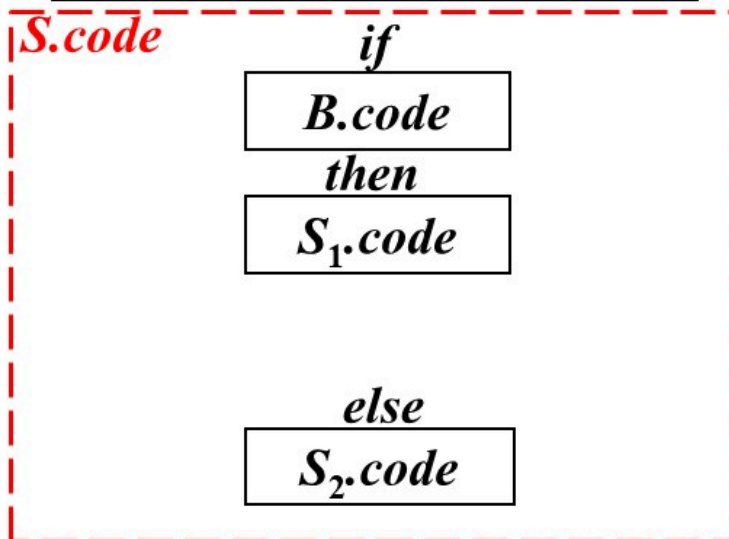
布尔表达式 B 被翻译成由
跳转指令构成的跳转代码

用指令的标号标识一条三地址指令

控制流语句的代码结构

➤ 例

$S \rightarrow \text{if } B \text{ then } S_1 \text{ else } S_2$



例1: $B = "a < b"$

三地址指令

if a < b goto *S₁.first*

goto *S₂.first*

临时指令

if a < b goto *B.true*

goto *B.false*

标号(常量)

变量

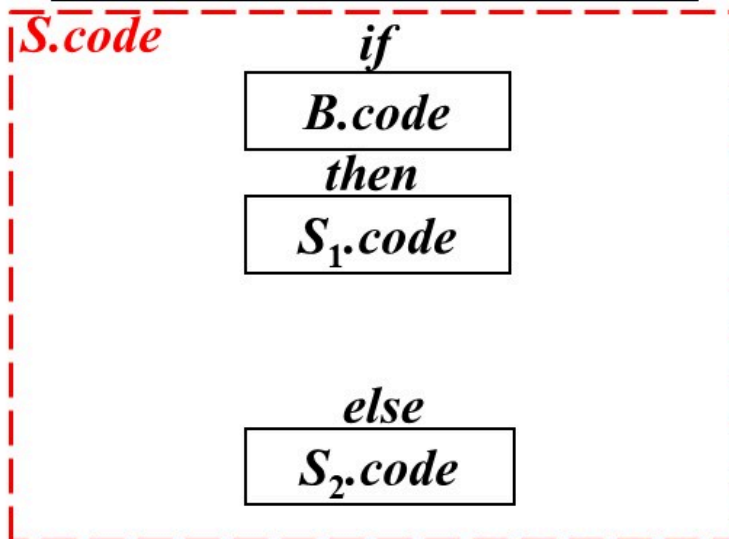
布尔表达式 B 被翻译成由
跳转指令构成的跳转代码

用指令的标号标识一条三地址指令

控制流语句的代码结构

➤ 例

$S \rightarrow \text{if } B \text{ then } S_1 \text{ else } S_2$



例1: B="a<b"

三地址指令

if a<b goto *S₁.first*

goto *S₂.first*

临时指令

if a<b goto *B.true*

goto *B.false*

if a<b goto (*B.true*)

goto (*B.false*)

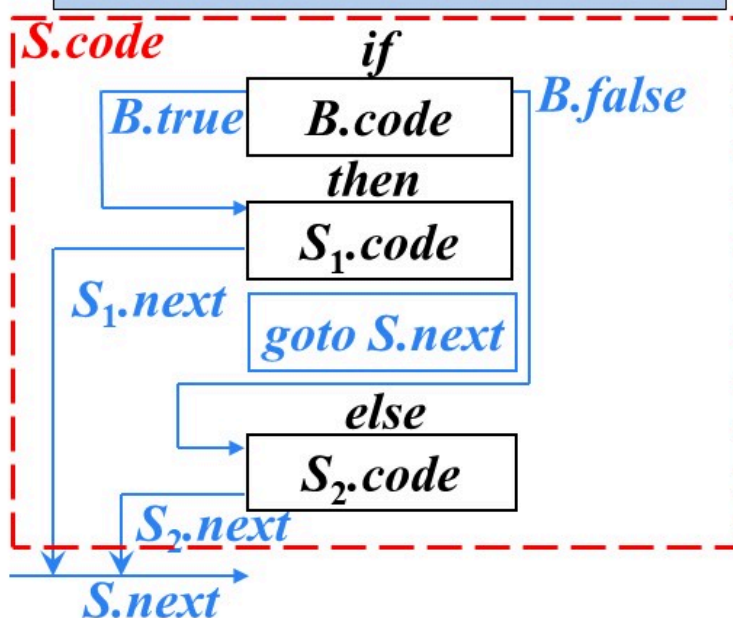
布尔表达式*B*被翻译成由
跳转指令构成的跳转代码

用指令的标号标识一条三地址指令

控制流语句的代码结构

➤ 例

$S \rightarrow \text{if } B \text{ then } S_1 \text{ else } S_2$



布尔表达式 B 被翻译成由
跳转指令构成的跳转代码

➤ 继承属性

- $B.true$: 是一个地址, 该地址用来存放当 B 为真时控制流转向的指令的标号
- $B.false$: 是一个地址, 该地址用来存放当 B 为假时控制流转向的指令的标号
- $S.next$: 是一个地址, 该地址用来存放紧跟在 S 代码之后执行的指令(S 的后继指令)的标号

用指令的标号标识一条三地址指令

控制流语句的SDT

newlabel(): 生成一个用于存放标号的新的临时变量L, 返回变量地址

➤ $P \rightarrow \{ S.next = newlabel(); \} S \{ label(S.next); \}$

➤ $S \rightarrow \{ S_1.next = newlabel(); \} S_1$

$\{ S_2.next = S.next; label(S_1.next); \} S_2$

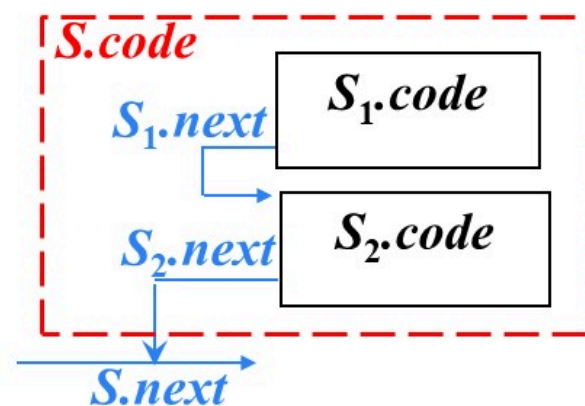
label(L): 将下一条三地址指令的标号存放到地址L中

➤ $S \rightarrow id = E ; \mid L = E ;$

➤ $S \rightarrow if\ B\ then\ S_1$

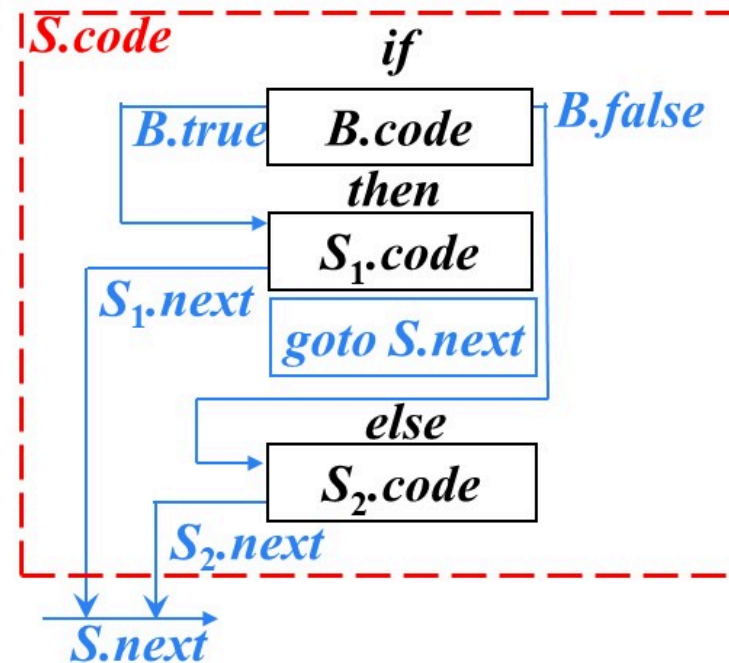
$\mid if\ B\ then\ S_1\ else\ S_2$

$\mid while\ B\ do\ S_1$



if-then-else语句的SDT

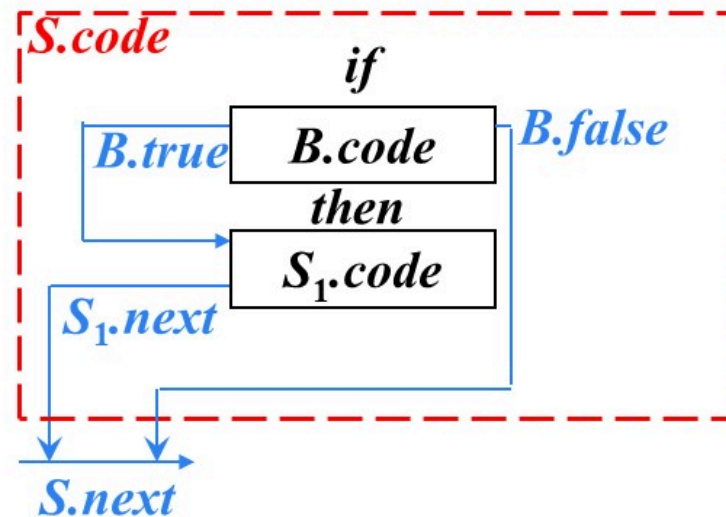
$S \rightarrow \text{if } B \text{ then } S_1 \text{ else } S_2$



$S \rightarrow \text{if } \{ B.true = \text{newlabel}(); B.false = \text{newlabel}(); \} B$
 $\text{then } \{ S_1.next = S.next; \text{label}(B.true); \} S_1 \{ \text{gen}('goto' S.next) \}$
 $\text{else } \{ S_2.next = S.next; \text{label}(B.false); \} S_2$

*if-then*语句的SDT

$S \rightarrow \text{if } B \text{ then } S_1$

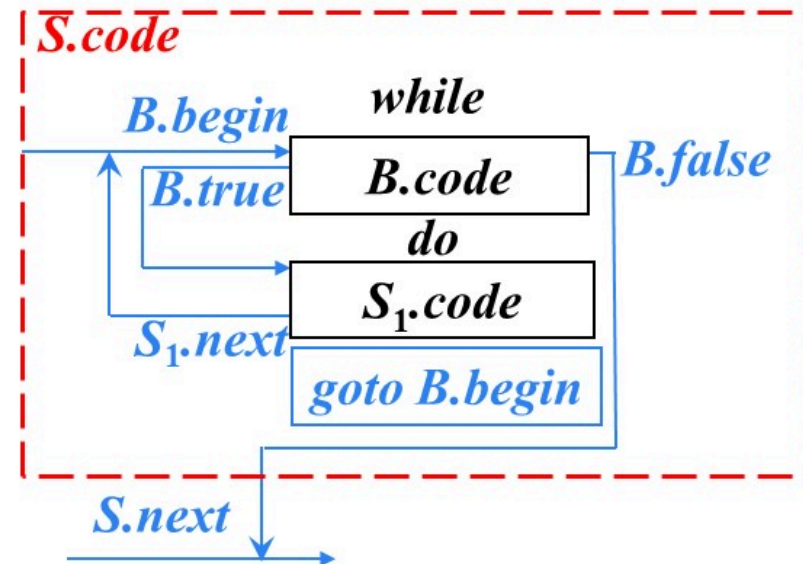


$S \rightarrow \text{if } \{ B.true = \text{newlabel}(); B.false = S.next; \} B$
 $\text{then } \{ S_1.next = S.next; \text{label}(B.true); \} S_1$

*while-do*语句的SDT

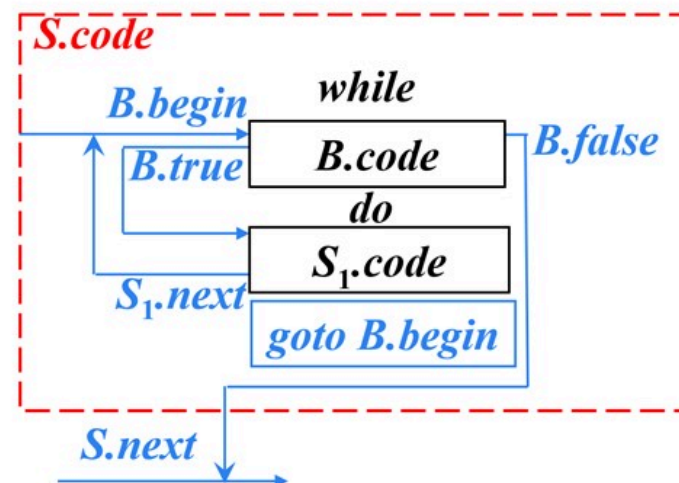
$S \rightarrow \text{while } B \text{ do } S_1$

$S \rightarrow \text{while } \{ B.true = \text{newlabel}();$
 $B.false = S.next;$
 $B.begin = \text{newlabel}();$
 $\text{label}(B.begin); \quad \} B$
 $\text{do } \{ S_1.next = B.begin; \text{label}(B.true); \} S_1$
 $\{ \text{gen}(\text{'goto' } B.begin); \}$



控制流语句SDT编写要点

- 分析每一个非终结符之前
 - 先计算继承属性
 - 再观察代码结构图中该非终结符对应的方框顶部是否有导入箭头。如果有，调用label()函数
- 上一个代码框执行完不顺序执行下一个代码框时，生成一条显式跳转指令
- 有自下而上的箭头时，设置begin属性。且定义后直接调用label()函数绑定地址



```
S → while { B.true = newlabel();
            B.false = S.next;
            B.begin = newlabel();
            label(B.begin); } B
do { S1.next = B.begin;
    label(B.true); } S1
{ gen('goto' B.begin); }
```

布尔表达式的翻译

➤ 布尔表达式的基本文法

$B \rightarrow B \text{ or } B$

| $B \text{ and } B$

| $\text{not } B$

| (B)

| $E \text{ relop } E$

| true

| false

逻辑运算符优先级: $\text{not} > \text{and} > \text{or}$

关系表达式

relop (关系运算符) :
 $<, <=, >, >=, ==, !=$

- 在跳转代码中，逻辑运算符**&&**、**||** 和 **!** 被翻译成**跳转指令**。运算符本身**不出现在代码中**，布尔表达式的值是通过代码序列中的位置来表示的

- 例

- 语句

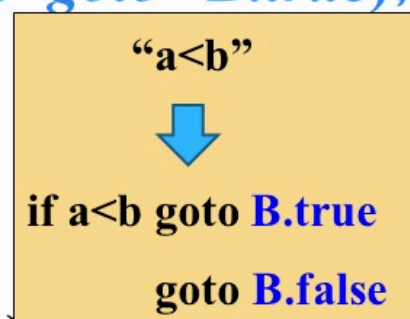
```
if ( x<100 || x>200 && x!=y )  
    x=0;
```

- 三地址代码

```
if x<100 goto L2  
goto L3  
L3 : if x>200 goto L4  
      goto L1  
L4 : if x!=y goto L2  
      goto L1  
L2 : x=0  
L1 :
```


布尔表达式的SDT

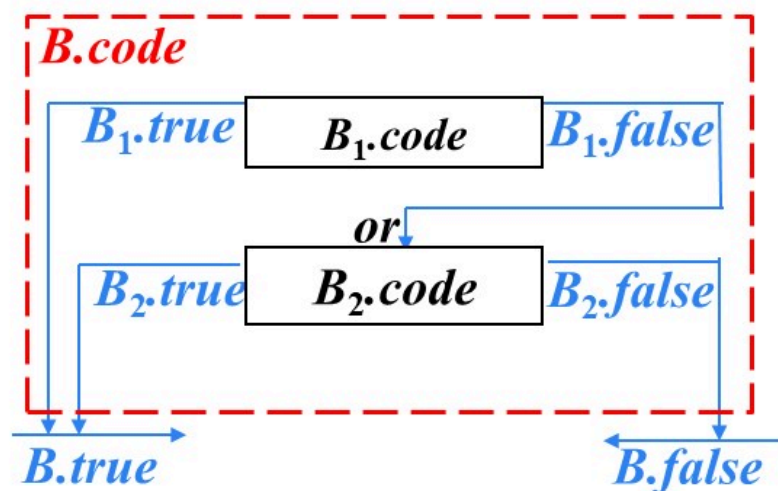
- $B \rightarrow E_1 \text{ relop } E_2 \{ \text{gen}(\text{'if' } E_1.\text{addr relop } E_2.\text{addr 'goto' } B.\text{true}); \text{gen}(\text{'goto' } B.\text{false}); \}$
- $B \rightarrow \text{true} \{ \text{gen}(\text{'goto' } B.\text{true}); \}$
- $B \rightarrow \text{false} \{ \text{gen}(\text{'goto' } B.\text{false}); \}$
- $B \rightarrow (\{ B_1.\text{true} = B.\text{true}; B_1.\text{false} = B.\text{false}; \} B_1)$
- $B \rightarrow \text{not} \{ B_1.\text{true} = B.\text{false}; B_1.\text{false} = B.\text{true}; \} B_1$
- $B \rightarrow B \text{ or } B$
- $B \rightarrow B \text{ and } B$



$B \rightarrow B_1 \text{ or } B_2$ 的SDT

➤ $B \rightarrow B_1 \text{ or } B_2$

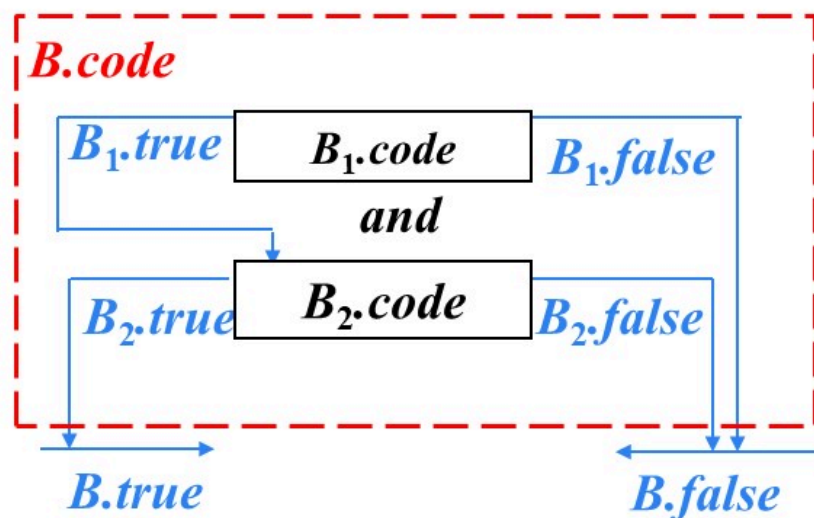
➤ $B \rightarrow \{ B_1.true = B.true; B_1.false = \text{newlabel}(); \} B_1$
or $\{ B_2.true = B.true; B_2.false = B.false; \text{label}(B_1.false); \} B_2$



$B \rightarrow B_1 \text{ and } B_2$ 的SDT

➤ $B \rightarrow B_1 \text{ and } B_2$

➤ $B \rightarrow \{ B_1.true = newlabel(); B_1.false = B.false; \} B_1$
and $\{ B_2.true = B.true; B_2.false = B.false; label(B_1.true); \} B_2$



控制流语句的 SDT

SDD: L-SDD

- 赋值语句：只定义了综合属性 (E.addr)
- 分支、循环语句：只定义了继承属性 (B.true、B.false、S.next)，且不依赖右兄弟节点属性值

- $P \rightarrow \{a\}S\{a\}$
- $S \rightarrow \{a\}S_1\{a\}S_2$
- $S \rightarrow id=E;\{a\} \mid L=E;\{a\}$
- $E \rightarrow E_1+E_2\{a\} \mid -E_1\{a\} \mid (E_1)\{a\} \mid id\{a\} \mid L\{a\}$
- $L \rightarrow id[E]\{a\} \mid L_1[E]\{a\}$ 赋值语句
- $S \rightarrow \text{if } \{a\}B \text{ then } \{a\}S_1$
 - | $\text{if } \{a\}B \text{ then } \{a\}S_1 \text{ else } \{a\}S_2$
 - | $\text{while } \{a\}B \text{ do } \{a\}S_1\{a\}$
- $B \rightarrow \{a\}B \text{ or } \{a\}B \mid \{a\}B \text{ and } \{a\}B \mid \text{not } \{a\}B \mid (\{a\}B)$
 - | $E \text{ relop } E\{a\} \mid \text{true}\{a\} \mid \text{false}\{a\}$

控制流语句的 SDT

- $P \rightarrow \{a\}S\{a\}$
- $S \rightarrow \{a\}S_1\{a\}S_2$
- $S \rightarrow id=E;\{a\} \mid L=E;\{a\}$
- $E \rightarrow E_1+E_2\{a\} \mid -E_1\{a\} \mid (E_1)\{a\} \mid id\{a\} \mid L\{a\}$
- $L \rightarrow id[E]\{a\} \mid L_1[E]\{a\}$
- $S \rightarrow \text{if } \{a\}B \text{ then } \{a\}S_1$
 - | $\text{if } \{a\}B \text{ then } \{a\}S_1 \text{ else } \{a\}S_2$
 - | $\text{while } \{a\}B \text{ do } \{a\}S_1\{a\}$
- $B \rightarrow \{a\}B \text{ or } \{a\}B \mid \{a\}B \text{ and } \{a\}B \mid \text{not } \{a\}B \mid (\{a\}B)$
 - | $E \text{ relop } E\{a\} \mid \text{true}\{a\} \mid \text{false}\{a\}$

SDD: L-SDD

- 赋值语句：只定义了综合属性 ($E.addr$)
- 分支、循环语句：只定义了继承属性 ($B.true$ 、 $B.false$ 、 $S.next$)，且不依赖右兄弟节点属性值

基础文法：可以使用LR分析技术

例：

```
while a < b do
  if c < d then
    x = y + z;
  else
    x = y - z;
```

*SDT*的实现方法

➤ *SDT*的通用实现方法

- 首先建立一棵语法分析树
- 然后按照从左到右的深度优先顺序来执行这些动作

控制流语句的SDT

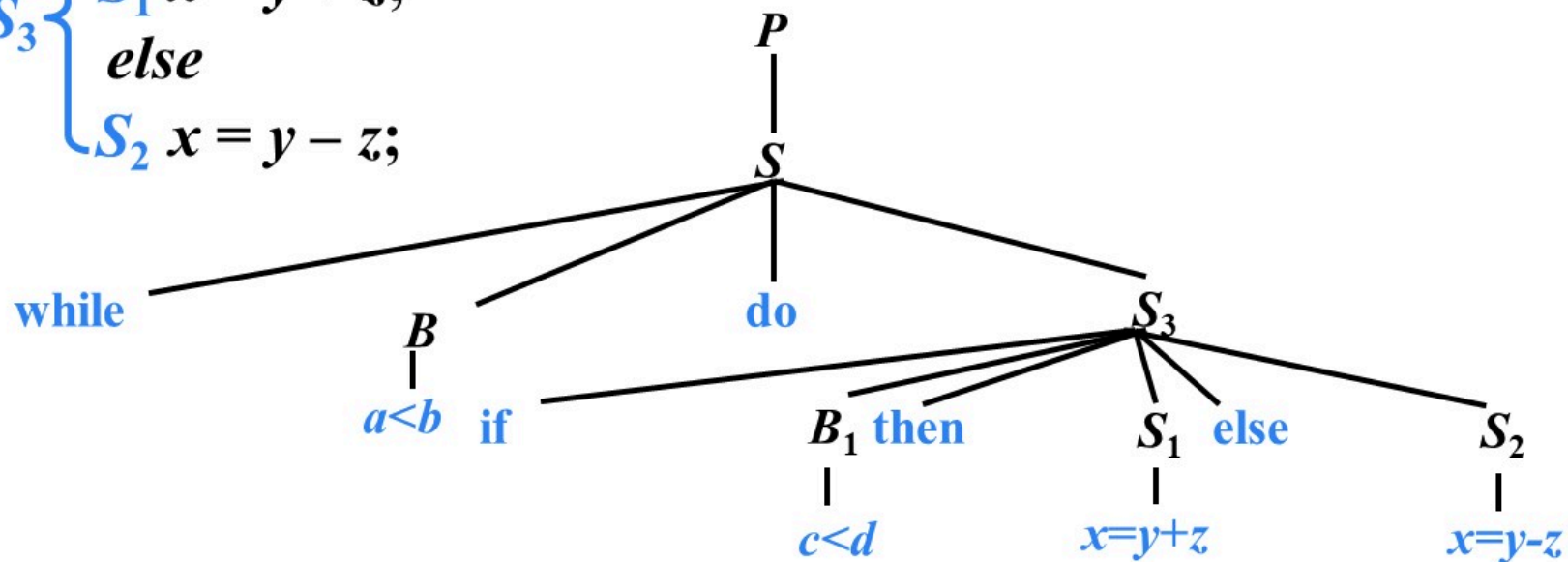
- $P \rightarrow \{a\}S\{a\}$
- $S \rightarrow \{a\}S_1\{a\}S_2$
- $S \rightarrow \text{id}=E;\{a\} \mid L=E;\{a\}$
- $E \rightarrow E_1+E_2\{a\} \mid -E_1\{a\} \mid (E_1)\{a\} \mid \text{id}\{a\} \mid L\{a\}$
- $L \rightarrow \text{id}[E]\{a\} \mid L_1[E]\{a\}$
- $S \rightarrow \text{if } \{a\}B \text{ then } \{a\}S_1$
 - | $\text{if } \{a\}B \text{ then } \{a\}S_1 \text{ else } \{a\}S_2$
 - | $\text{while } \{a\}B \text{ do } \{a\}S_1\{a\}$
- $B \rightarrow \{a\}B \text{ or } \{a\}B \mid \{a\}B \text{ and } \{a\}B \mid \text{not } \{a\}B \mid (\{a\}B)$
 - | $E \text{ relop } E\{a\} \mid \text{true}\{a\} \mid \text{false}\{a\}$

例：

```
while a < b do
  if c < d then
    x = y + z;
  else
    x = y - z;
```

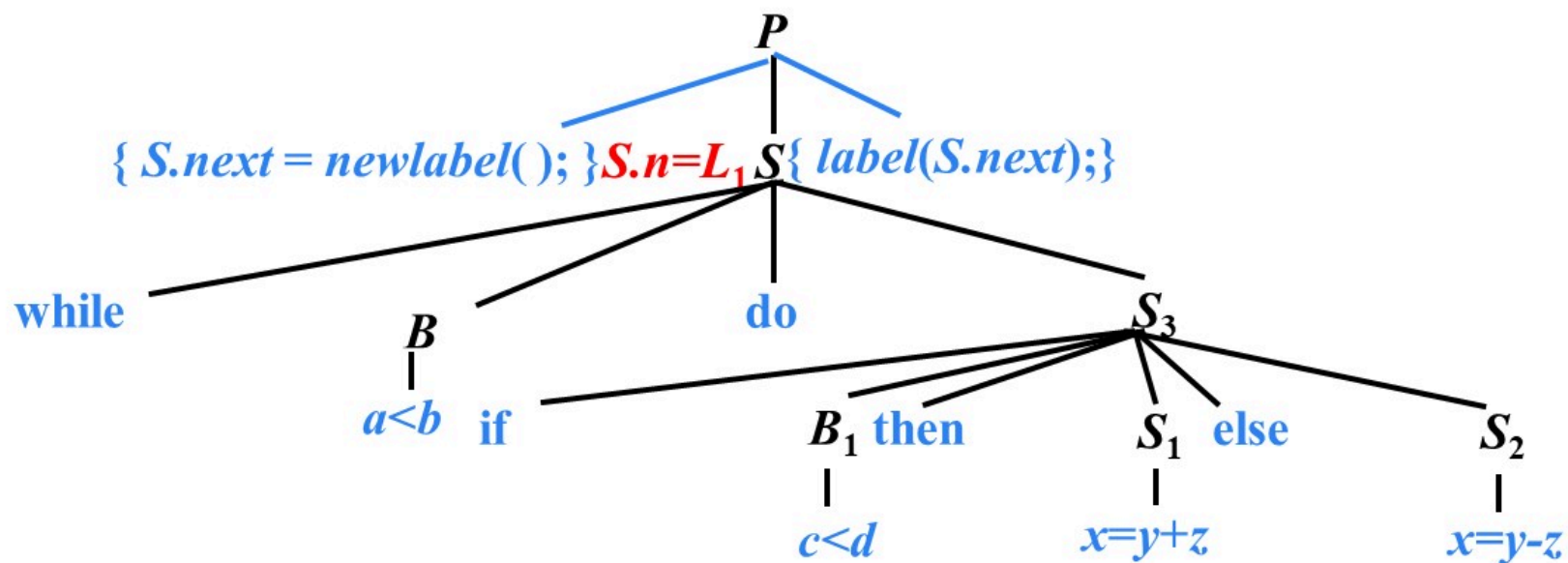
例

$\text{while } [a < b] \text{ do}$
 $\left\{ \begin{array}{l} \text{if } [c < d] \text{ then} \\ S_1 \ x = y + z; \\ \text{else} \\ S_2 \ x = y - z; \end{array} \right.$



例

$P \rightarrow \{ S.next = newlabel(); \} S \{ label(S.next); \}$



例

$S \rightarrow \text{while } \{B.\text{begin} = \text{newlabel}();$

$\text{label}(B.\text{begin});$

$B.\text{true} = \text{newlabel}();$

$B.\text{false} = S.\text{next}; \} B$

$\text{do } \{ \text{label}(B.\text{true});$

$S_1.\text{next} = B.\text{begin}; \} S_1$

$\{ \text{gen}(\text{'goto' } B.\text{begin}); \}$

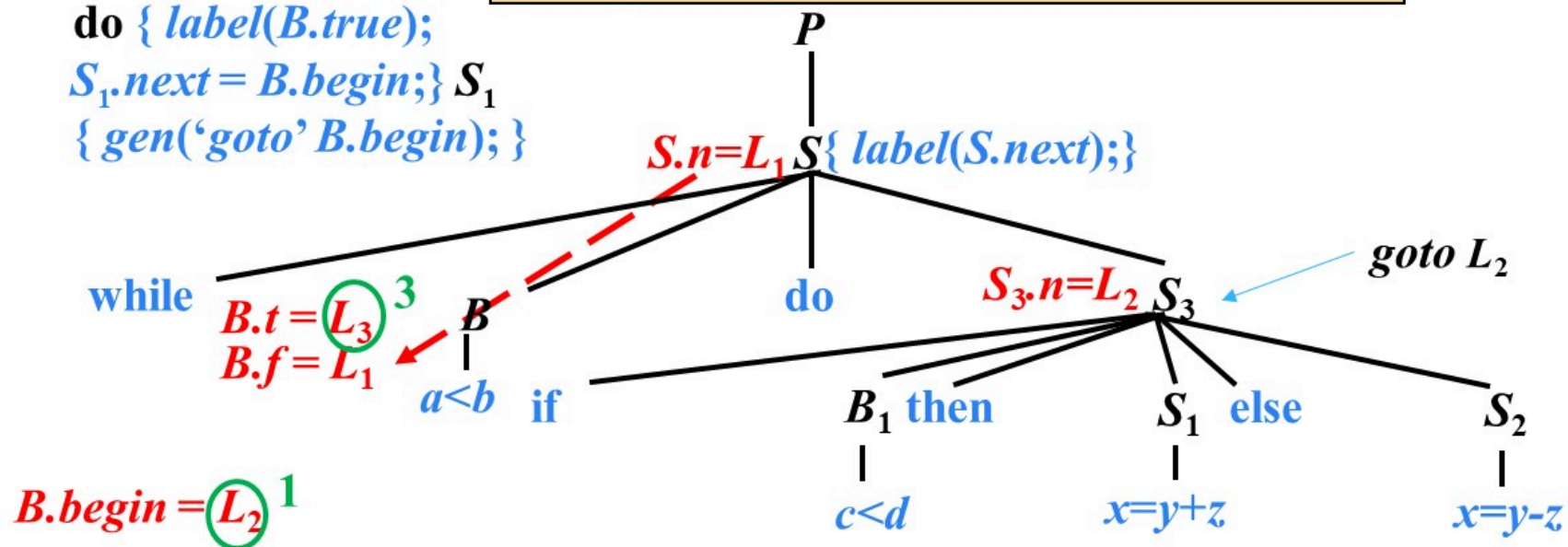
$B \rightarrow E_1 \text{ relop } E_2$

$\{ \text{gen}(\text{'if' } E_1.\text{addr relop } E_2.\text{addr 'goto' } B.\text{true});$

$\text{gen}(\text{'goto' } B.\text{false}); \}$

1: $\text{if } a < b \text{ goto } L_3$

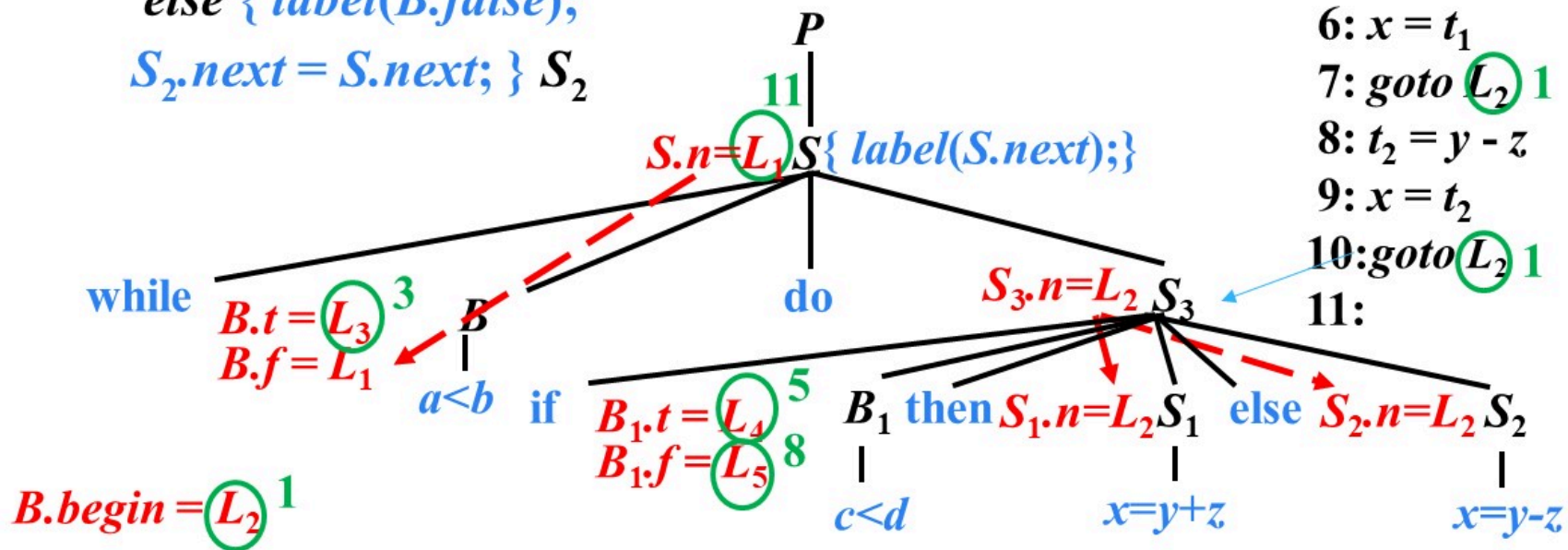
2: $\text{goto } L_1$



例

$S \rightarrow \text{if } \{ B.\text{true} = \text{newlabel}(); B.\text{false} = \text{newlabel}(); \} B$
 $\text{then } \{ \text{label}(B.\text{true}); S_1.\text{next} = S.\text{next}; \} S_1$
 $\{ \text{gen}('goto' S.\text{next}); \}$
 $\text{else } \{ \text{label}(B.\text{false});$
 $S_2.\text{next} = S.\text{next}; \} S_2$

1: if $a < b$ goto L_3
 2: goto L_1 11
 3: if $c < d$ goto L_4
 4: goto L_5 8
 5: $t_1 = y + z$
 6: $x = t_1$
 7: goto L_2 1
 8: $t_2 = y - z$
 9: $x = t_2$
 10: goto L_2 1
 11:



上一讲内容回顾

- 6.1 声明语句的翻译
- 6.2 赋值语句的翻译
- 6.3 控制语句的翻译
 - $P \rightarrow S$
 - $S \rightarrow S_1 S_2$
 - $S \rightarrow \text{id} = E ; \mid L = E ;$
 - $S \rightarrow \text{if } B \text{ then } S_1$
 - | $\text{if } B \text{ then } S_1 \text{ else } S_2$
 - | $\text{while } B \text{ do } S_1$

布尔表达式 B 被翻译成由跳转指令构成的跳转代码

问题：生成跳转指令的时候，目标标号的值可能还不知道

语句 “*while* $a < b$ *do if* $c < d$ *then* $x = y + z$ *else* $x = y - z$ ” 的三地址代码

1: <i>if</i> $a < b$ <i>goto</i> 3	1: ($j <$, a , b , 3)
2: <i>goto</i> 11	2: (j , -, -, 11)
3: <i>if</i> $c < d$ <i>goto</i> 5	3: ($j <$, c , d , 5)
4: <i>goto</i> 8	4: (j , -, -, 8)
5: $t_1 = y + z$	5: (+, y , z , t_1)
6: $x = t_1$	6: (=, t_1 , -, x)
7: <i>goto</i> 1	7: (j , -, -, 1)
8: $t_2 = y - z$	8: (-, y , z , t_2)
9: $x = t_2$	9: (=, t_2 , -, x)
10: <i>goto</i> 1	10: (j , -, -, 1)
11:	11:

语句 “*while a < b do if c < d then x = y + z else x = y - z*” 的三地址代码

B.first → 1: *if a < b goto 3*

2: *goto 11*

S₁.first → 3: *if c < d goto 5*

4: *goto 8*

5: $t_1 = y + z$

6: $x = t_1$

7: *goto 1*

8: $t_2 = y - z$

9: $x = t_2$

10: *goto 1*

11:

1: *if False a < b goto 11*

2:

3: *if c < d goto 5*

4: *goto 8*

5: $t_1 = y + z$

6: $x = t_1$

7: *goto 1*

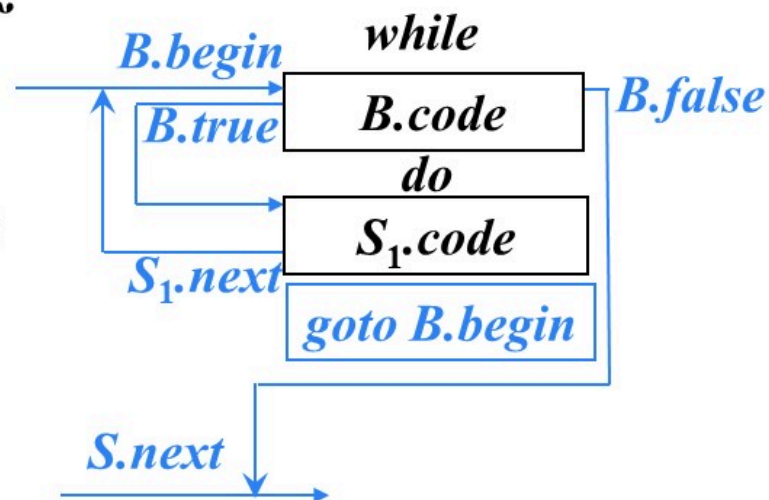
8: $t_2 = y - z$

9: $x = t_2$

10: *goto 1*

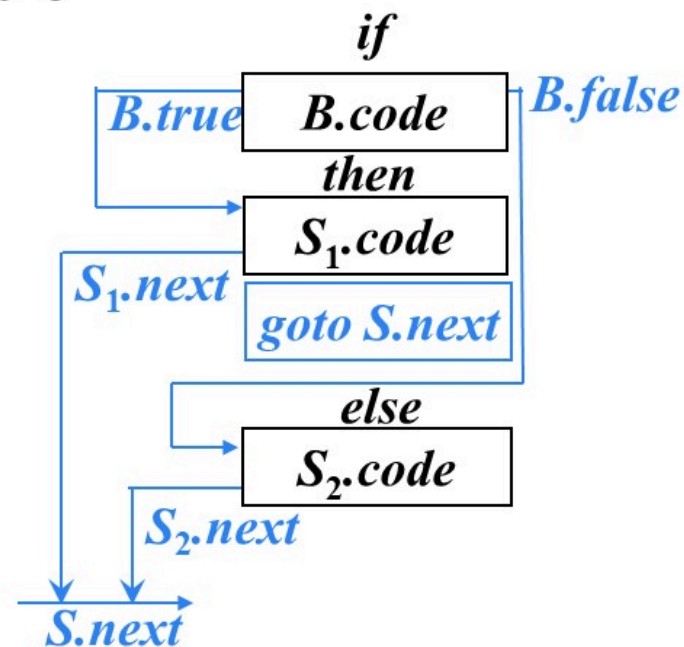
11:

避免生成冗余的
goto指令



语句 “*while a < b do if c < d then x = y + z else x = y - z*” 的三地址代码

1: <i>if a < b goto 3</i>	1: <i>ifFalse a < b goto 11</i>
2: <i>goto 11</i>	2:
<i>B.first</i> → 3: <i>if c < d goto 5</i>	3: <i>if c < d goto 5</i>
4: <i>goto 8</i>	4: <i>goto 8</i>
<i>S₁.first</i> → 5: <i>t₁ = y + z</i>	5: <i>t₁ = y + z</i>
6: <i>x = t₁</i>	6: <i>x = t₁</i>
7: <i>goto 1</i>	7: <i>goto 1</i>
<i>S₂.first</i> → 8: <i>t₂ = y - z</i>	8: <i>t₂ = y - z</i>
9: <i>x = t₂</i>	9: <i>x = t₂</i>
10: <i>goto 1</i>	10: <i>goto 1</i>
11:	11:



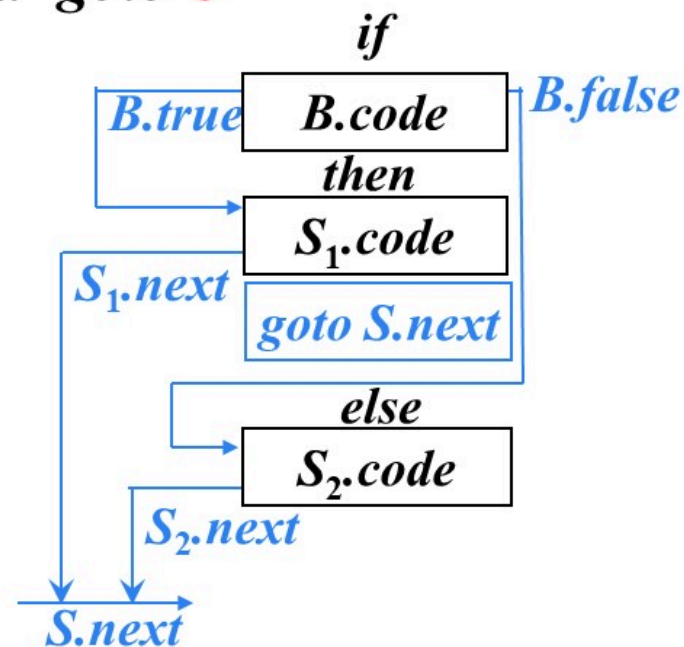
语句 “*while a < b do if c < d then x = y + z else x = y - z*” 的三地址代码

```

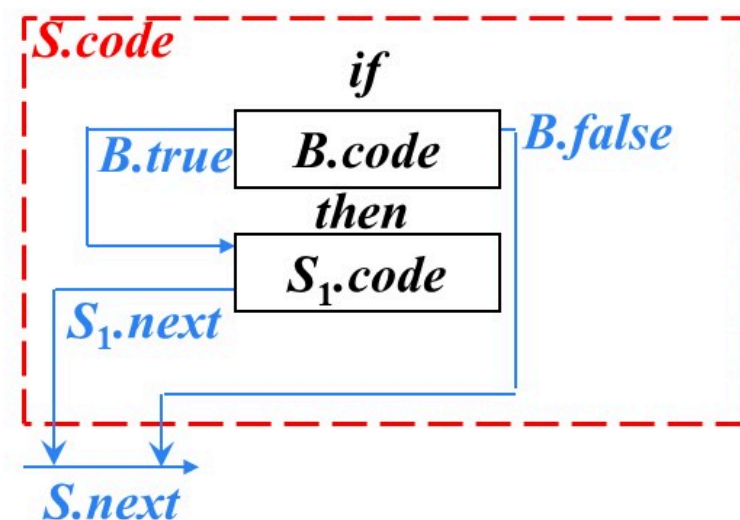
1: if a < b goto 3
2: goto 11
B.first → 3: if c < d goto 5
4: goto 8
S1.first → 5: t1 = y + z
6: x = t1
7: goto 1
S2.first → 8: t2 = y - z
9: x = t2
10: goto 1
11:
    
```

```

1: if False a < b goto 11
2:
3: if False c < d goto 8
4:
5: t1 = y + z
6: x = t1
7: goto 1
8: t2 = y - z
9: x = t2
10: goto 1
11:
    
```



修改 *if-then* 语句的 *SDT*

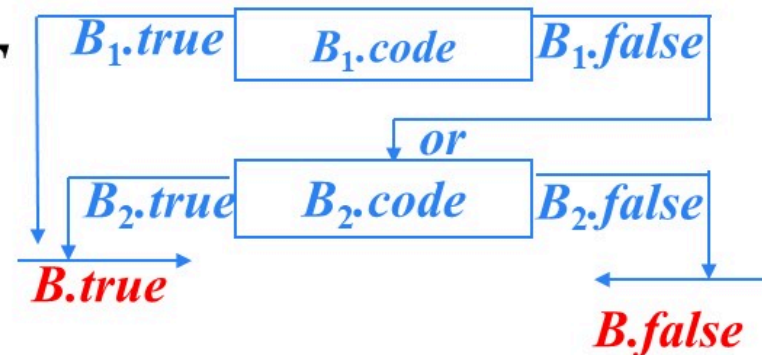


$S \rightarrow \text{if } \{ B.true = \text{newlabel}(); B.false = S.next; \} B$
 $\text{then } \{ S_1.next = S.next; \text{label}(B.true); \} S_1$

省略。不生成任何跳转指令

$S \rightarrow \text{if } \{ B.true = \text{fall}; B.false = S.next; \} B$
 $\text{then } \{ S_1.next = S.next; \} S_1$

修改 $B \rightarrow B_1 \text{ or } B_2$ 的SDT

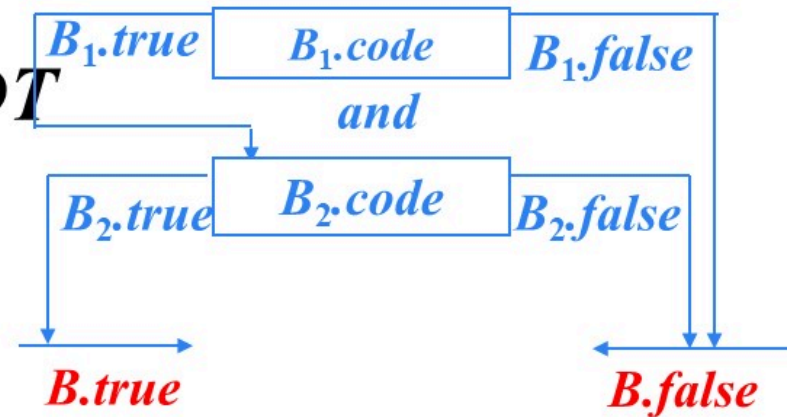


$B \rightarrow \{ B_1.true = B.true; B_1.false = newlabel(); \} B_1$
 or $\{ B_2.true = B.true; B_2.false = B.false; label(B_1.false); \} B_2$



$B \rightarrow \{ B_1.true = B.true == fall ? newlabel() : B.true; B_1.false = fall; \} B_1$ or
 $\{ B_2.true = B.true; B_2.false = B.false; \} B_2 \{ \text{if } B.true == fall \text{ then } label(B_1.true); \}$

修改 $B \rightarrow B_1 \text{ and } B_2$ 的SDT



$B \rightarrow \{ B_1.true = newlabel(); B_1.false = B.false; \} B_1$
 and $\{ B_2.true = B.true; B_2.false = B.false; label(B_1.true); \} B_2$



$B \rightarrow \{ B_1.true = fall; B_1.false = B.false == fall ? newlabel(): B.false; \} B_1$
 and $\{ B_2.true = B.true; B_2.false = B.false; \} B_2$
 $\{ if B.false == fall then label(B_1.false); \}$

修改 $B \rightarrow E_1 \text{ relop } E_2$ 的SDT

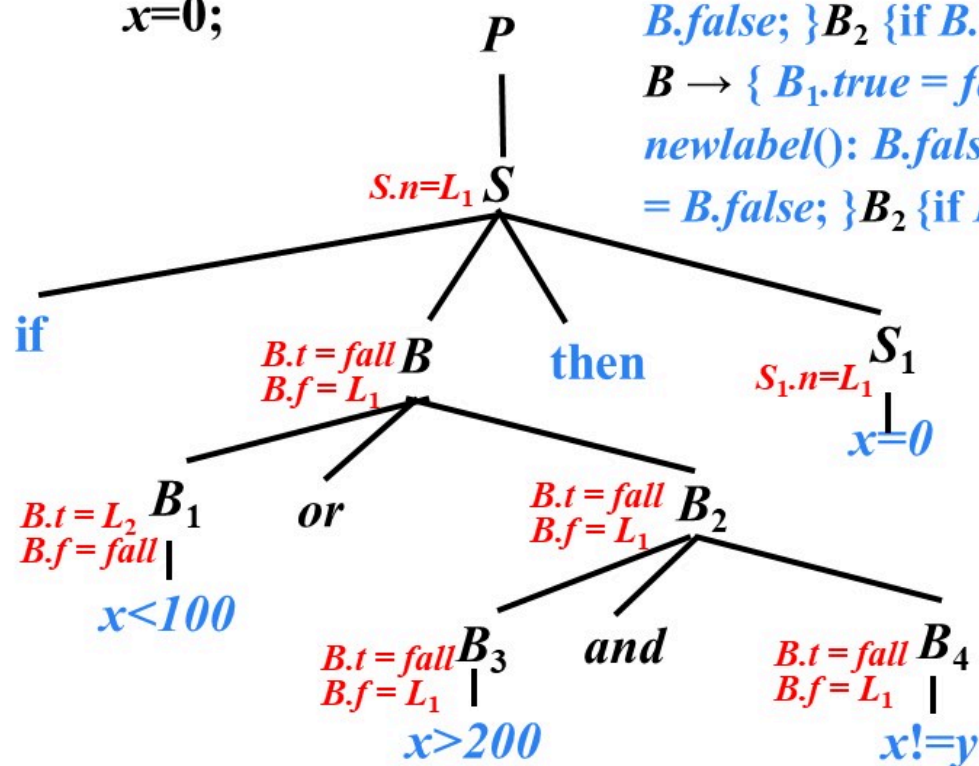
➤ $B \rightarrow E_1 \text{ relop } E_2 \{ \text{gen('if' } E_1.\text{addr relop } E_2.\text{addr 'goto' } B.\text{true});$
 $\text{gen('goto' } B.\text{false}); \}$



➤ $B \rightarrow E_1 \text{ relop } E_2$
{ if $B.\text{true} \neq \text{fall}$ and $B.\text{false} \neq \text{fall}$ then
 $\text{gen('if' } E_1.\text{addr relop } E_2.\text{addr 'goto' } B.\text{true}); \text{ gen('goto' } B.\text{false});$
 else if $B.\text{true} \neq \text{fall}$ then $\text{gen('if' } E_1.\text{addr relop } E_2.\text{addr 'goto' } B.\text{true});$
 else if $B.\text{false} \neq \text{fall}$ then $\text{gen('ifFalse' } E_1.\text{addr relop } E_2.\text{addr 'goto' } B.\text{false});$
 else '' }

例

if ($x < 100 \parallel x > 200 \ \&\& \ x \neq y$)
 $x = 0$;



$S \rightarrow \text{if} \{ B.\text{true} = \text{fall}; B.\text{false} = S.\text{next}; \} B$
 $\text{then} \{ S_1.\text{next} = S.\text{next}; \} S_1$

$B \rightarrow \{ B_1.\text{true} = B.\text{true} == \text{fall} ? \text{newlabel}(): B.\text{true};$
 $B_1.\text{false} = \text{fall}; \} B_1$ **or** $\{ B_2.\text{true} = B.\text{true}; B_2.\text{false} =$
 $B.\text{false}; \} B_2 \{ \text{if } B.\text{true} == \text{fall} \text{ then label}(B_1.\text{true}); \}$

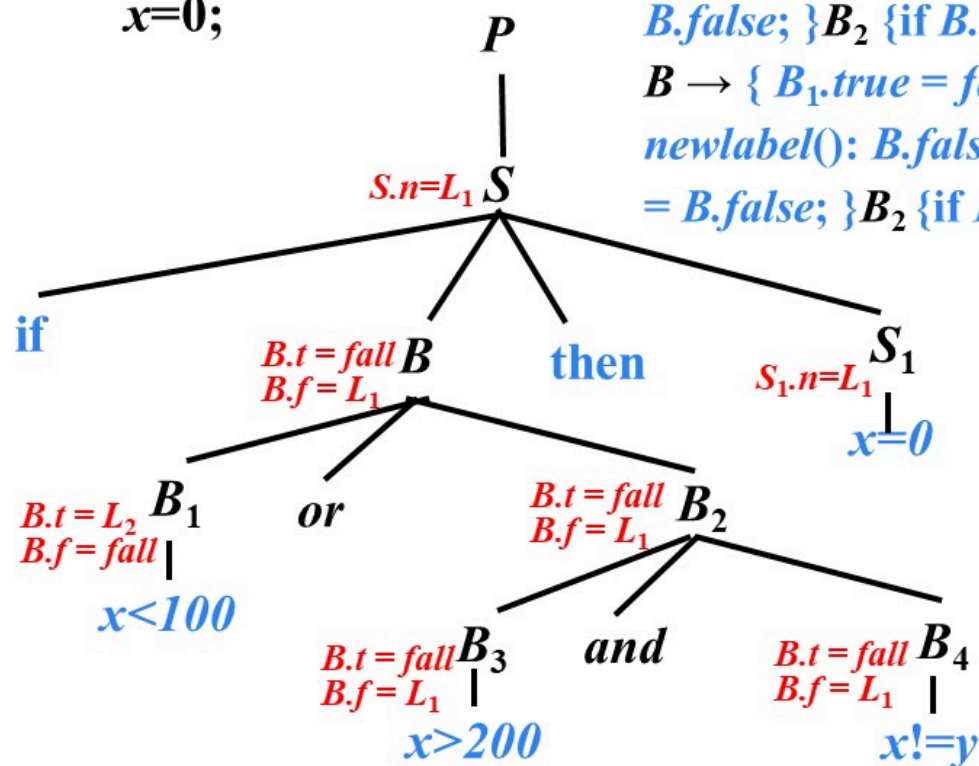
$B \rightarrow \{ B_1.\text{true} = \text{fall}; B_1.\text{false} = B.\text{false} == \text{fall} ?$
 $\text{newlabel}(): B.\text{false}; \} B_1$ **and** $\{ B_2.\text{true} = B.\text{true}; B_2.\text{false} =$
 $B.\text{false}; \} B_2 \{ \text{if } B.\text{false} == \text{fall} \text{ then label}(B_1.\text{false}); \}$

if $x < 100$ *goto* L_2
ifFalse $x > 200$ *goto* L_1
ifFalse $x \neq y$ *goto* L_1

$L_2: \ x = 0$

例

if ($x < 100 \parallel x > 200 \ \&\& \ x \neq y$)
 $x = 0$;



$S \rightarrow \text{if} \{ B.\text{true} = \text{fall}; B.\text{false} = S.\text{next}; \} B$
 $\text{then} \{ S_1.\text{next} = S.\text{next}; \} S_1$

$B \rightarrow \{ B_1.\text{true} = B.\text{true} == \text{fall} ? \text{newlabel}(): B.\text{true};$
 $B_1.\text{false} = \text{fall}; \} B_1$ **or** $\{ B_2.\text{true} = B.\text{true}; B_2.\text{false} =$
 $B.\text{false}; \} B_2 \{ \text{if } B.\text{true} == \text{fall} \text{ then label}(B_1.\text{true}); \}$

$B \rightarrow \{ B_1.\text{true} = \text{fall}; B_1.\text{false} = B.\text{false} == \text{fall} ?$
 $\text{newlabel}(): B.\text{false}; \} B_1$ **and** $\{ B_2.\text{true} = B.\text{true}; B_2.\text{false} =$
 $B.\text{false}; \} B_2 \{ \text{if } B.\text{false} == \text{fall} \text{ then label}(B_1.\text{false}); \}$

if $x < 100$ *goto* L_2
ifFalse $x > 200$ *goto* L_1
ifFalse $x \neq y$ *goto* L_1

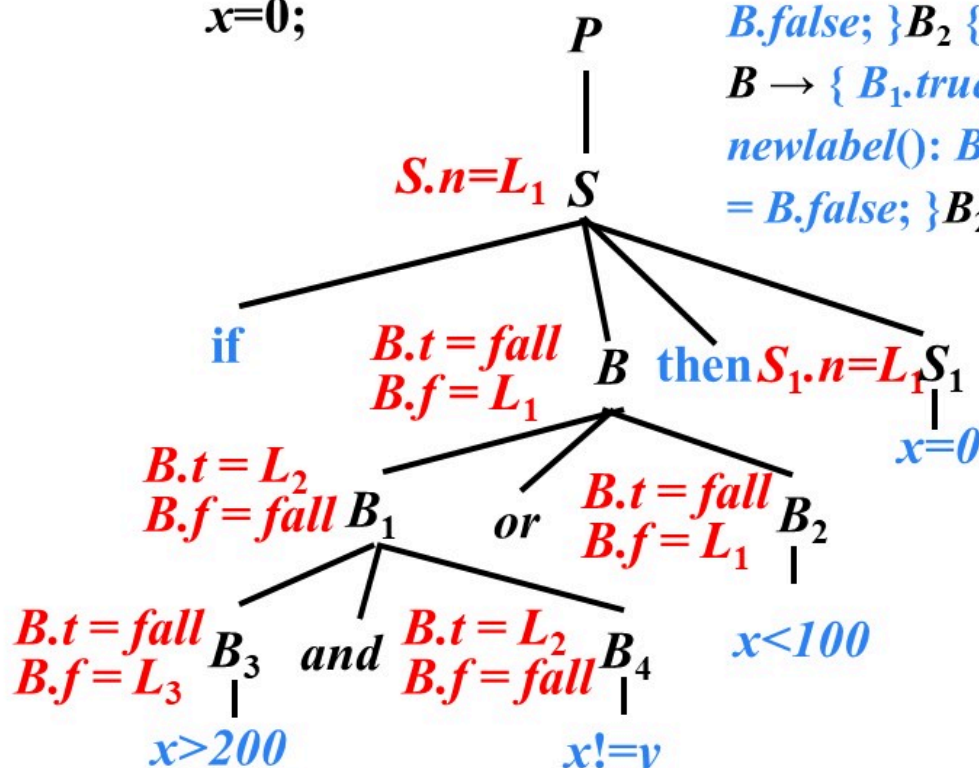
$L_2: \quad x = 0$

$L_1:$

例

if (*x*>200 && *x*!=*y* || *x*<100)

x=0;



$S \rightarrow \text{if} \{ B.\text{true} = \text{fall}; B.\text{false} = S.\text{next}; \} B$

$\text{then } \{ S_1.\text{next} = S.\text{next}; \} S_1$

$B \rightarrow \{ B_1.\text{true} = B.\text{true} == \text{fall} ? \text{newlabel}(): B.\text{true};$
 $B_1.\text{false} = \text{fall}; \} B_1$ **or** $\{ B_2.\text{true} = B.\text{true}; B_2.\text{false} =$
 $B.\text{false}; \} B_2 \{ \text{if } B.\text{true} == \text{fall} \text{ then label}(B_1.\text{true}); \}$

$B \rightarrow \{ B_1.\text{true} = \text{fall}; B_1.\text{false} = B.\text{false} == \text{fall} ?$
 $\text{newlabel}(): B.\text{false}; \} B_1$ **and** $\{ B_2.\text{true} = B.\text{true}; B_2.\text{false} =$
 $B.\text{false}; \} B_2 \{ \text{if } B.\text{false} == \text{fall} \text{ then label}(B_1.\text{false}); \}$

ifFalse *x*>200 goto *L*₃

if *x*!=*y* goto *L*₂

*L*₃ : *ifFalse* *x*<100 goto *L*₁

*L*₂ : *x*=0

*L*₁ :



提纲

6.1 声明语句的翻译

6.2 赋值语句的翻译

6.3 控制语句的翻译

6.4 回填

6.5 switch语句的翻译

6.6 过程调用语句的翻译

6.4 回填 (Backpatching)

➤ 基本思想

- 生成一个跳转指令时，暂时不指定该跳转指令的目标标号。这样的指令都被放入由跳转指令组成的列表中。同一个列表中的所有跳转指令具有相同的目标标号。等到能够确定正确的目标标号时，才去填充这些指令的目标标号

非终结符 B 的综合属性

- $B.truelist$: 指向一个包含跳转指令的列表, 这些指令最终获得的目标标号就是当 B 为真时控制流应该转向的指令的标号
- $B.falselist$: 指向一个包含跳转指令的列表, 这些指令最终获得的目标标号就是当 B 为假时控制流应该转向的指令的标号

将 B 中跳往 B 的真出口 (即 B 的值为真时前往的指令) 和假出口 (即 B 的值为假时前往的指令) 的指令分别放入 $B.truelist$ 和 $B.falselist$

函数

➤ *makelist(i)*

- 创建一个只包含*i*的列表，*i*是跳转指令的标号，函数返回指向新创建的列表的指针

➤ *merge(p₁, p₂)*

- 将 *p₁* 和 *p₂* 指向的列表进行合并，返回指向合并后的列表的指针

➤ *backpatch(p, i)*

- 将 *i* 作为目标标号插入到 *p* 所指列表中的各指令中

布尔表达式的回填

➤ $B \rightarrow E_1 \text{ relop } E_2$

{

B.truelist = makelist(nextquad); *nextquad*: 即将生成的下一条指令的标号

B.falselist = makelist(nextquad+1);

gen('if' $E_1.addr \text{ relop } E_2.addr$ 'goto _');

gen('goto _');

}

布尔表达式的回填

➤ $B \rightarrow E_1 \text{ relop } E_2$

➤ $B \rightarrow \text{true}$

```
{  
    B.truelist = makelist(nextquad);  
    gen('goto _');  
}
```


布尔表达式的回填

➤ $B \rightarrow E_1 \text{ relop } E_2$

➤ $B \rightarrow \text{true}$

➤ $B \rightarrow \text{false}$

```
{  
    B.falselist = makelist(nextquad);  
    gen('goto _');  
}
```

布尔表达式的回填

➤ $B \rightarrow E_1 \text{ relop } E_2$

➤ $B \rightarrow \text{true}$

➤ $B \rightarrow \text{false}$

➤ $B \rightarrow (B_1)$

{

$B.\text{truelist} = B_1.\text{truelist} ;$

$B.\text{falselist} = B_1.\text{falselist} ;$

}

布尔表达式的回填

➤ $B \rightarrow E_1 \text{ relop } E_2$

➤ $B \rightarrow \text{true}$

➤ $B \rightarrow \text{false}$

➤ $B \rightarrow (B_1)$

➤ $B \rightarrow \text{not } B_1$

{

$B.\text{truelist} = B_1.\text{falselist};$

$B.\text{falselist} = B_1.\text{truelist};$

}

 $\{$
$$B.falselist = B_2.falselist ;$$

}

```
{ M.quad = nextquad ; }
```



$B \rightarrow B_1 \text{ and } B_2$

$B \rightarrow B_1 \text{ and } M B_2$

{

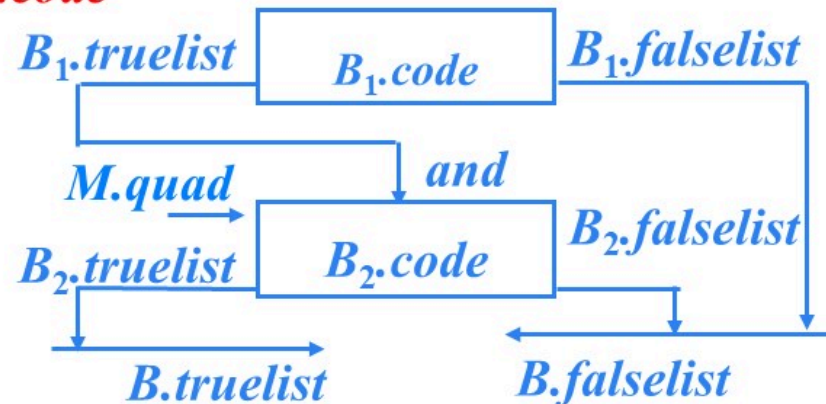
$B.truelist = B_2.truelist;$

$B.falselist = merge(B_1.falselist, B_2.falselist);$

$backpatch(B_1.truelist, M.quad);$

}

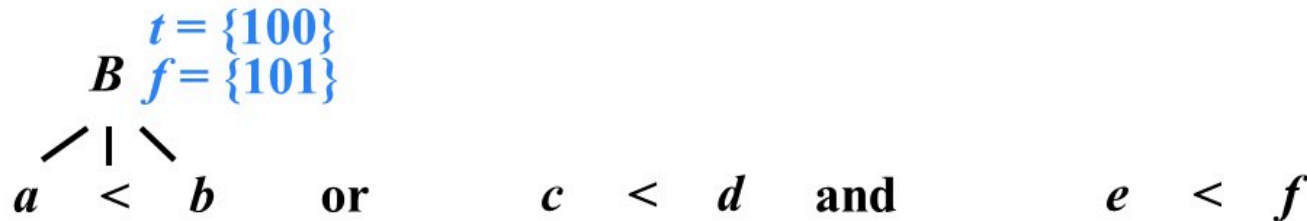
S.code



例

```
 $B \rightarrow E_1 \text{ relop } E_2$   
{  $B.\text{truelist} = \text{makelist}(\text{nextquad});$   
   $B.\text{falselist} = \text{makelist}(\text{nextquad}+1);$   
   $\text{gen}(\text{'if' } E_1.\text{addr relop } E_2.\text{addr 'goto _'});$   
   $\text{gen}(\text{'goto _'});$   
}
```

```
100:  $\text{if } a < b \text{ goto } \_$   
101:  $\text{goto } \_$ 
```



例

$B \rightarrow B_1 \text{ or } M B_2$

{

$\text{backpatch}(B_1.\text{falselist}, M.\text{quad});$

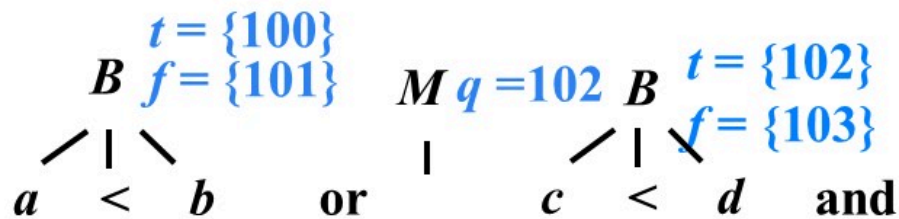
$B.\text{truelist} = \text{merge}(B_1.\text{truelist}, B_2.\text{truelist});$

$B.\text{falselist} = B_2.\text{falselist};$

}

$M \rightarrow \varepsilon$

{ $M.\text{quad} = \text{nextquad};$ }

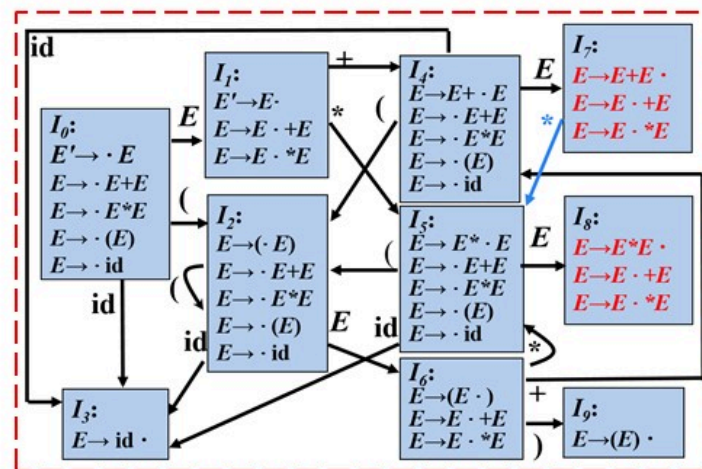


100: if $a < b$ goto _

101: goto _

102: if $c < d$ goto _

103: goto _



例

$B \rightarrow B_1 \text{ and } M B_2$

{

$B.\text{truelist} = B_2.\text{truelist};$

$B.\text{falselist} = \text{merge}(B_1.\text{falselist}, B_2.\text{falselist});$

$\text{backpatch}(B_1.\text{truelist}, M.\text{quad});$

}

100: *if* $a < b$ *goto* _

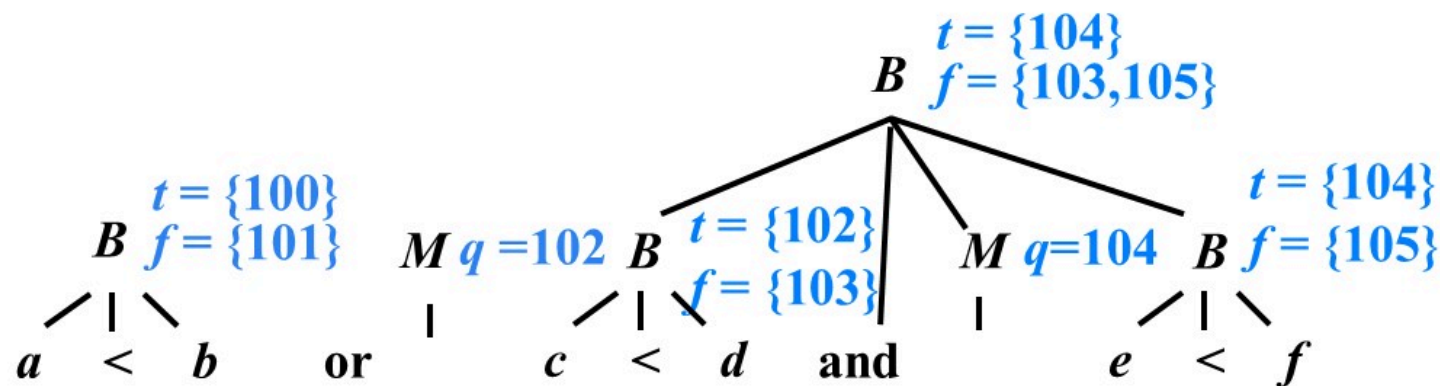
101: *goto* _

102: *if* $c < d$ *goto* 104

103: *goto* _

104: *if* $e < f$ *goto* _

105: *goto* _



例

$B \rightarrow B_1 \text{ or } M B_2$

{

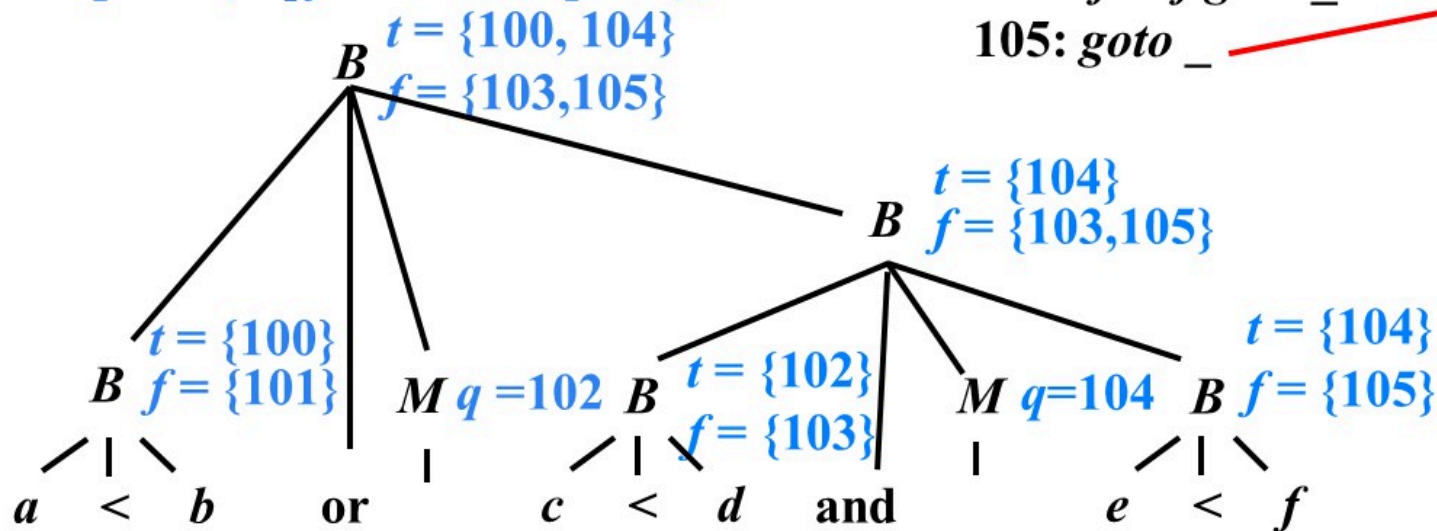
$B.\text{truelist} = \text{merge}(B_1.\text{truelist}, B_2.\text{truelist});$

$B.\text{falselist} = B_2.\text{falselist};$

$\text{backpatch}(B_1.\text{falselist}, M.\text{quad});$

}

100: if $a < b$ goto $_$ t
 101: goto $\underline{102}$
 102: if $c < d$ goto $\underline{104}$
 103: goto $_$
 104: if $e < f$ goto $_$ f
 105: goto $_$



控制流语句的回填

➤ 文法

➤ $S \rightarrow S_1 S_2$

➤ $S \rightarrow \text{id} = E ; \mid L = E ;$

➤ $S \rightarrow \text{if } B \text{ then } S_1$
 $\mid \text{if } B \text{ then } S_1 \text{ else } S_2$
 $\mid \text{while } B \text{ do } S_1$

➤ 综合属性

➤ $S.nextlist$: 指向一个包含跳转指令的列表, 这些指令最终获得的目标标号就是按照运行顺序紧跟在S代码之后的指令的标号

$S \rightarrow \text{if } B \text{ then } S_1$

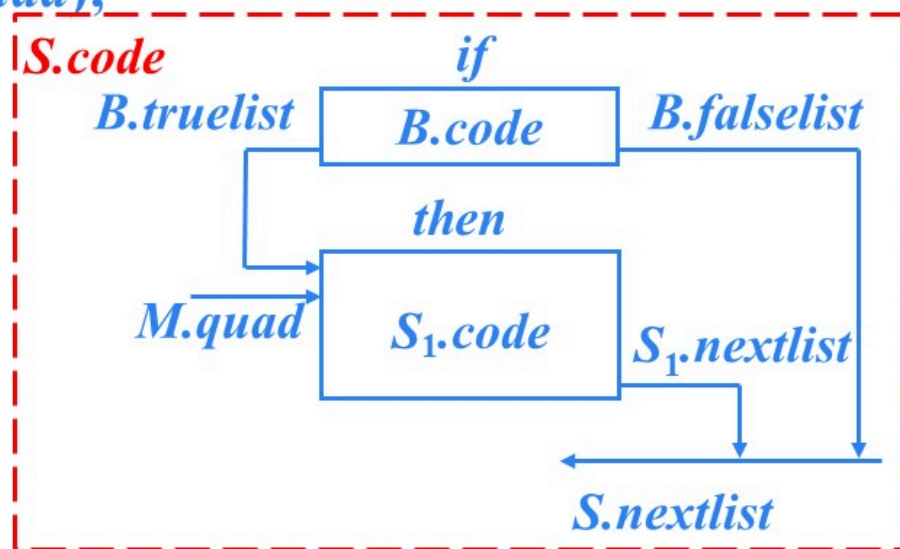
$S \rightarrow \text{if } B \text{ then } M S_1$

{

$S.\text{nextlist} = \text{merge}(B.\text{falselist}, S_1.\text{nextlist});$

$\text{backpatch}(B.\text{truelist}, M.\text{quad});$

}



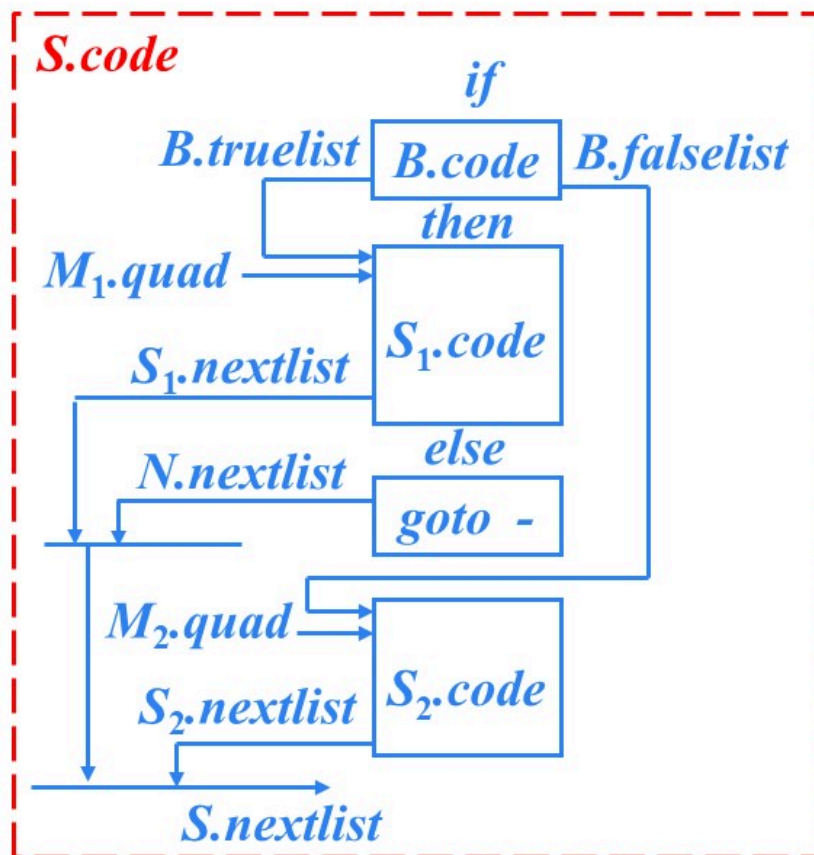
$S \rightarrow \text{if } B \text{ then } S_1 \text{ else } S_2$

$S \rightarrow \text{if } B \text{ then } M_1 S_1 \text{ else } N M_2 S_2$

```
{
  S.nextlist = merge( merge(S1.nextlist,
    N.nextlist), S2.nextlist );
  backpatch(B.truelist, M1.quad);
  backpatch(B.falselist, M2.quad);
}
```

$N \rightarrow \varepsilon$

```
{
  N.nextlist = makelist(nextquad);
  gen('goto _');
}
```

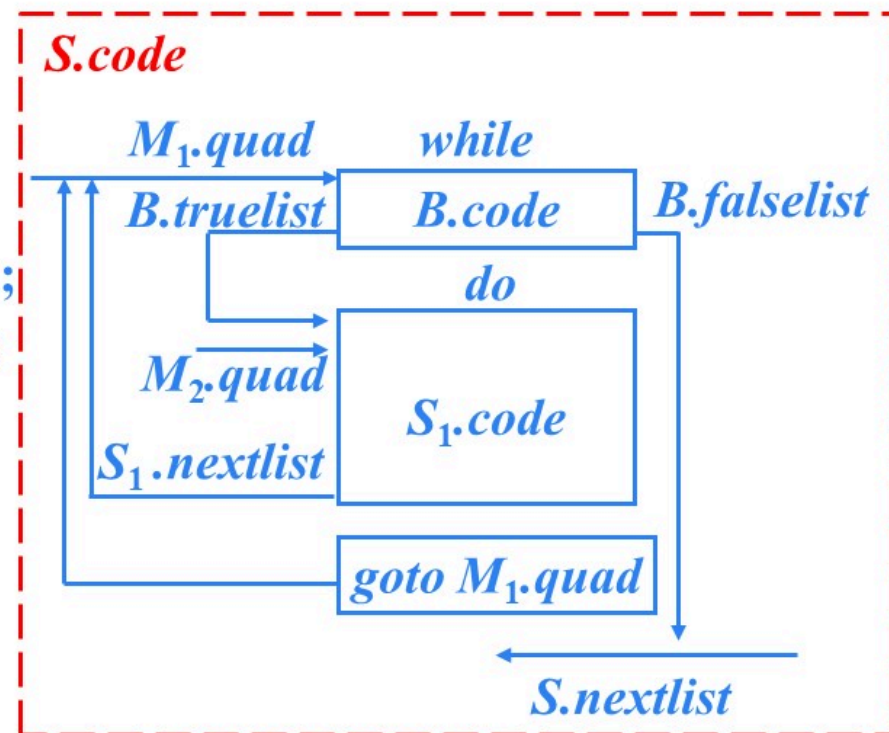


$S \rightarrow \text{while } B \text{ do } S_1$

```

S  $\rightarrow$  while M1 B do M2 S1
{
    S.nextlist = B.falselist;
    backpatch( S1.nextlist, M1.quad );
    backpatch( B.truelist, M2.quad );
    gen('goto' M1.quad);
}

```



$S \rightarrow S_1 S_2$

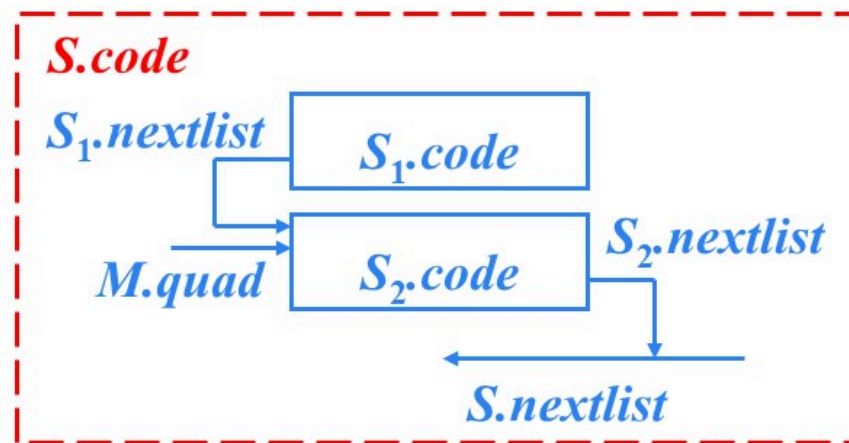
$S \rightarrow S_1 M S_2$

{

$S.nextlist = S_2.nextlist;$

$backpatch(S_1.nextlist, M.quad);$

}

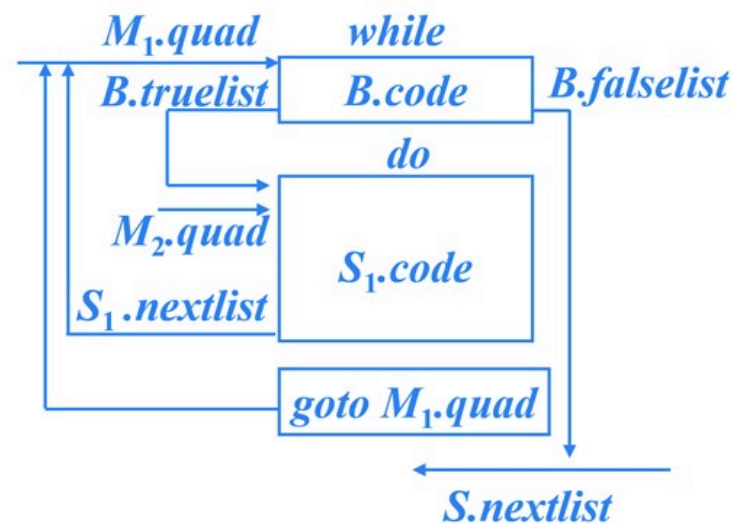


 $S \rightarrow \text{id} = E ; \mid L = E ;$

$S \rightarrow \text{id} = E ; \mid L = E ; \{ S.nextlist = null; \}$

回填技术SDT编写要点

- 语法改造
 - 在list箭头指向的位置设置标记非终结符M
- 在产生式末尾的语义动作中
 - 计算综合属性
 - 调用backpatch()函数回填各个list
 - 生成必要的附加指令



```

S → while M1 B do M2 S1
{
    S.nextlist = B.falselist;
    backpatch( S1.nextlist, M1.quad );
    backpatch( B.true, M2.quad );
    gen('goto' M1.quad);
}
    
```

 例

while a < b do

if c < 5 then

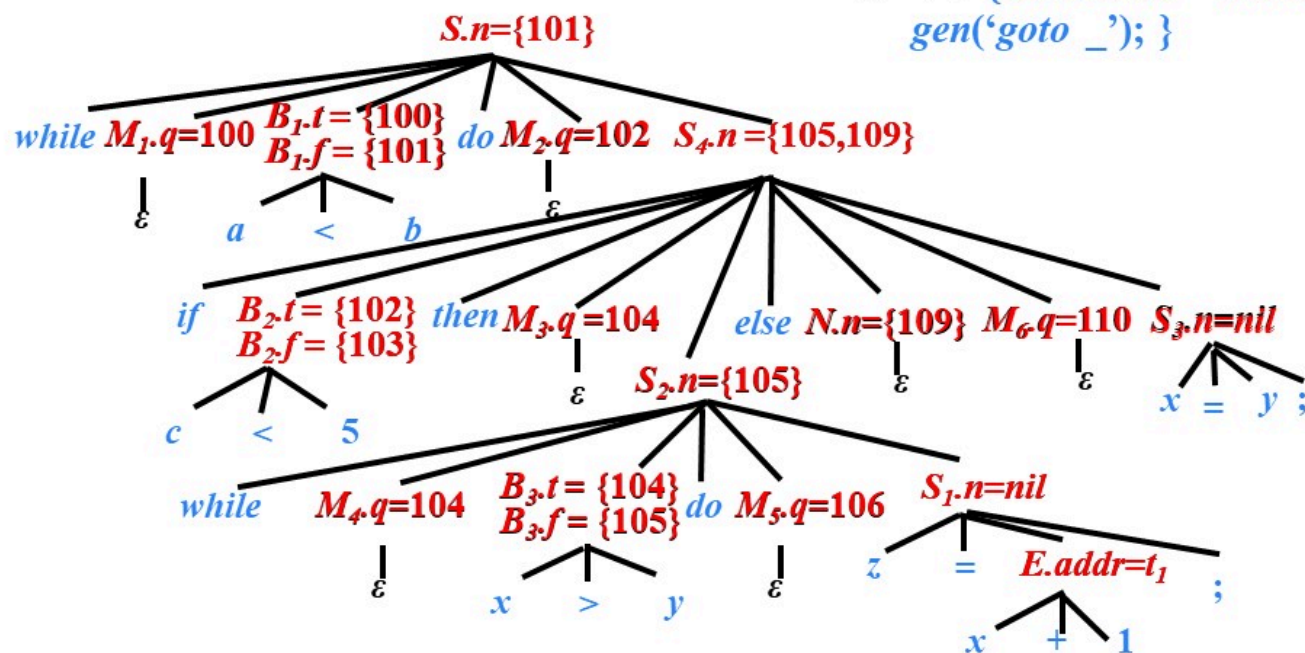
while x > y do z = x + 1;

else

x = y;

$S \rightarrow \text{while } M_1 B \text{ do } M_2 S_1$
 $\{ S.\text{nextlist} = B.\text{falselist};$
 $\text{backpatch}(S_1.\text{nextlist}, M_1.\text{quad});$
 $\text{backpatch}(B.\text{truelist}, M_2.\text{quad});$
 $\text{gen}(\text{'goto'} M_1.\text{quad}); \}$

$S \rightarrow \text{if } B \text{ then } M_1 S_1 N \text{ else } M_2 S_2$
 $\{ S.\text{nextlist} = \text{merge}(\text{merge}(S_1.\text{nextlist}, N.\text{nextlist}),$
 $S_2.\text{nextlist});$
 $\text{backpatch}(B.\text{truelist}, M_1.\text{quad});$
 $\text{backpatch}(B.\text{falselist}, M_2.\text{quad}); \}$
 $N \rightarrow \epsilon \{ N.\text{nextlist} = \text{makelist}(\text{nextquad});$
 $\text{gen}(\text{'goto'} _); \}$



100: if $a < b$ goto 102
 101: goto _
 102: if $c < 5$ goto 104
 103: goto 110
 104: if $x > y$ goto 106
 105: goto 100
 106: $t_1 = x + 1$
 107: $z = t_1$
 108: goto 104
 109: goto 100
 110: $x = y$
 111: goto 100
 112:

语句 “ $\text{while } a < b \text{ do if } c < 5 \text{ then while } x > y \text{ do } z = x + 1; \text{ else } x = y;$ ” 的注释分析树

while a < b do if c < 5 then while x > y do z = x + 1; else x = y;

while 100: <i>if a < b goto 102</i>	100: (j<, a, b, 102)
101: <i>goto _</i>	101: (j, -, -, -)
if 102: <i>if c < 5 goto 104</i>	102: (j<, c, 5, 104)
103: <i>goto 110</i>	103: (j, -, -, 110)
while 104: <i>if x > y goto 106</i>	104: (j>, x, y, 106)
105: <i>goto 100</i>	105: (j, -, -, 100)
106: $t_1 = x + 1$	106: (+, x, 1, t_1)
107: $z = t_1$	107: (=, t_1 , -, z)
108: <i>goto 104</i> ← 来自于内层 while 语句	108: (j, -, -, 104)
109: <i>goto 100</i> ← 来自于 if 语句	109: (j, -, -, 100)
110: $x = y$	110: (=, y, -, x)
111: <i>goto 100</i> ← 来自于外层 while 语句	111: (j, -, -, 100)
112:	112:



提纲

6.1 声明语句的翻译

6.2 赋值语句的翻译

6.3 控制语句的翻译

6.4 回填

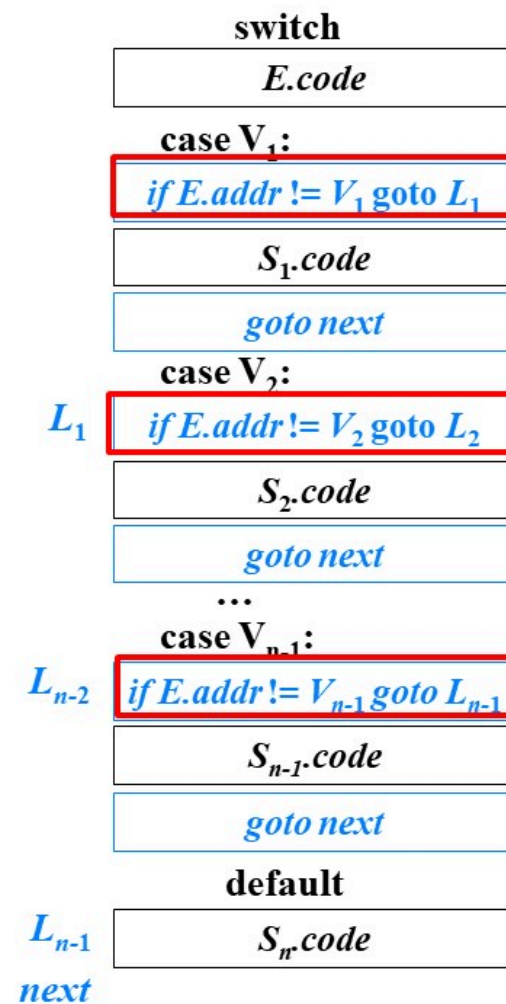
6.5 switch语句的翻译

6.6 过程调用语句的翻译

6.5 *switch*语句的翻译

```

switch E
begin
    case  $V_1$ :  $S_1$ 
    case  $V_2$ :  $S_2$ 
    ...
    case  $V_{n-1}$ :  $S_{n-1}$ 
    default:  $S_n$ 
end
    
```

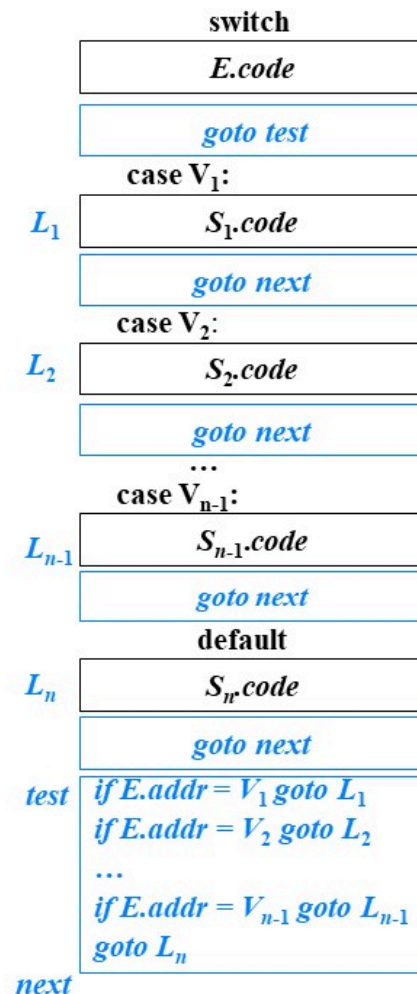


switch语句的另一种翻译

```

switch E
begin
    case  $V_1$ :  $S_1$ 
    case  $V_2$ :  $S_2$ 
    ...
    case  $V_{n-1}$ :  $S_{n-1}$ 
    default:  $S_n$ 
end
    
```

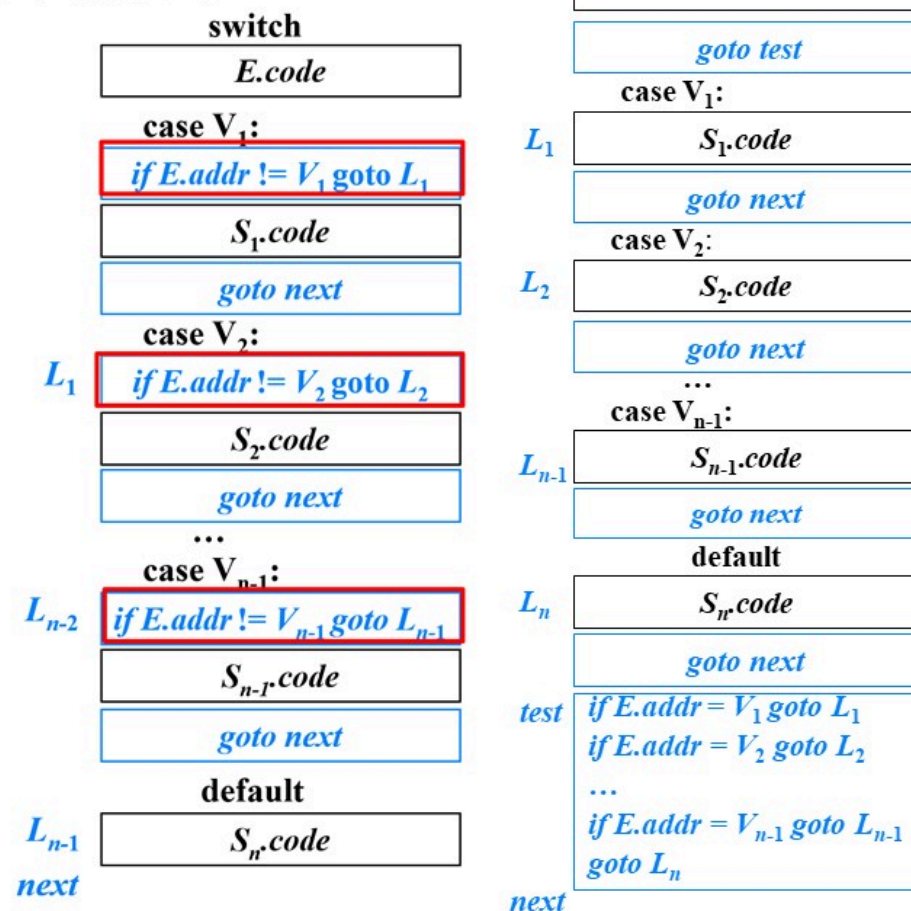
在代码生成阶段，根据分支的个数以及这些值是否在一个较小的范围内，这种条件跳转指令序列可以被翻译成最高效的 n 路分支



switch语句的另一种翻译

```

switch E
begin
  case V1: S1
  case V2: S2
  ...
  case Vn-1: Sn-1
  default: Sn
end
  
```



增加一种 *case* 指令

```
test :  if  $t = V_1$  goto  $L_1$ 
        if  $t = V_2$  goto  $L_2$ 
        ...
        if  $t = V_{n-1}$  goto  $L_{n-1}$ 
        goto  $L_n$ 
next :
```

```
test :  case  $t$   $V_1$   $L_1$ 
        case  $t$   $V_2$   $L_2$ 
        ...
        case  $t$   $V_{n-1}$   $L_{n-1}$ 
        case  $t$   $t$   $L_n$ 
next :
```

指令 *case* t V_i L_i 和 *if* $t = V_i$ goto L_i 的含义相同，
但是 *case* 指令更加容易被最终的代码生成器探测到，
从而对这些指令进行特殊处理



提纲

- 6.1 声明语句的翻译
- 6.2 赋值语句的翻译
- 6.3 控制语句的翻译
- 6.4 回填
- 6.5 switch语句的翻译
- 6.6 过程调用语句的翻译**

6.6 过程调用的翻译

➤ 文法

➤ $S \rightarrow \text{call id } (Elist)$

$Elist \rightarrow Elist, E$

$Elist \rightarrow E$

过程调用语句的代码结构

$\text{call id}(E_1, E_2, \dots, E_n)$

$\text{call id} ($

$E_1.\text{code}$

$\text{param } E_1.\text{addr}$

,

$E_2.\text{code}$

$\text{param } E_2.\text{addr}$

,

$\dots$

,

$E_n.\text{code}$

$\text{param } E_n.\text{addr}$

)

$\text{call id.addr } n$

$\text{call id} ($

$E_1.\text{code}$

,

$E_2.\text{code}$

,

$\dots$

,

$E_n.\text{code}$

)

$\text{param } E_1.\text{addr}$

$\text{param } E_2.\text{addr}$

$\dots$

$\text{param } E_n.\text{addr}$

$\text{call id.addr } n$

过程调用语句的代码结构

$\text{call id}(E_1, E_2, \dots, E_n)$

$\text{call id} ($

$E_1.\text{code}$

,

$E_2.\text{code}$

,

$\dots$

,

$E_n.\text{code}$

)

需要一个队列 q 存放 $E_1.\text{addr}$ 、 $E_2.\text{addr}$ 、 $\dots$ 、 $E_n.\text{addr}$ ，以生成

$\text{param } E_1.\text{addr}$
 $\text{param } E_2.\text{addr}$
 $\dots$
 $\text{param } E_n.\text{addr}$
 $\text{call id.addr } n$

过程调用语句的SDD

➤ $S \rightarrow \text{call id} (Elist)$

```
{
     $n=0$ ;
    for  $q$  中的每个  $t$  do
    {
         $\text{gen}(\text{'param' } t)$ ;
         $n = n+1$ ;
    }
     $\text{gen}(\text{'call' id.addr}, n)$ ;
}
```

➤ $Elist \rightarrow E$

```
{ 将  $q$  初始化为只包含  $E.addr$ ; }
```

➤ $Elist \rightarrow Elist_1, E$

```
{ 将  $E.addr$  添加到  $q$  的队尾; }
```

call id (

$E_1.code$

,

$E_2.code$

,

...

,

$E_n.code$

)

param $E_1.addr$

param $E_2.addr$

...

param $E_n.addr$

call id.addr n

 例：翻译以下语句 $f(b * c - 1, x + y, x, y)$

$t_1 = b * c$

$t_2 = t_1 - 1$

$t_3 = x + y$

param t_2

param t_3

param x

param y

call $f, 4$

本章小结

- 声明语句的翻译
- 赋值语句的翻译
 - 简单赋值语句的翻译
 - 数组引用的翻译
- 控制语句的翻译
- 回填
- switch语句的翻译
- 过程调用语句的翻译

作业

- 预习/自学“长江雨课堂”平台第7章（运行存储分配）中的以下视频
 - 7-1 运行存储分配概述
 - 7-2 静态存储分配
 - 7-3 栈式存储分配
 - 7-4 调用序列和返回序列



结束

